

PETsys
Electronics

TOFPET2 ASIC Evaluation Kit

Software user guide v2025.07

Contents

1	System Overview	6
1.1	TOFPET2 ASIC Evaluation Kit	6
1.1.1	Programming the FEB/D FPGA	6
1.2	Computer requirements	7
1.3	Software	7
1.4	Before first use	8
2	Quickstart	10
2.1	Starting the GUI	10
2.1.1	Launching “daqd”	11
2.2	Configuration and calibration	12
2.3	Data Acquisition with a radioactive ^{22}Na point source	13
2.3.1	Coincidence events and Coincidence Time Resolution	14
3	Reference Configuration File and Calibration procedure	16
3.1	Setting the working data folder, HV bias and mapping tables	18
3.1.1	Mapping tables	20
3.2	Generating ASIC calibration files	21
3.2.1	Discriminator calibration	22
3.2.2	TDC calibration	22
3.2.3	QDC calibration	22
4	Data acquisition	26
4.1	Data acquisition template	26
4.2	Data acquisition procedure	27
4.2.1	Enabling only a subset of ASICs for acquisition	29
4.2.2	Parameter scan	30
4.2.3	Hardware coincidence trigger	30
4.2.4	Multi-hit group and energy sum filter	31
4.2.5	Coincidence filtering	32
4.2.6	Software simulation	32
4.3	Manual or internally triggered acquisition start	33
4.4	Temperature reading and stability	33
4.5	Other utility applications	34
4.6	Using an external trigger	35

5	Data processing	36
5.1	Singles application	36
5.2	Group application	38
5.2.1	Compact data formats	40
5.3	Coincidence application	42
5.3.1	Compact data formats	44
5.4	Event filtering parameters	46
5.5	Additional output options	46
5.6	Hardware trigger software simulation	47
5.7	Time offset (or skew) calibration	47
5.8	QDC charge measurement linearisation / Energy Calibration (Optional)	48
5.9	Raw to raw application	49
5.10	Data Processing with GUI	50
5.11	Real-time data processing	50
A	Absolute ID of Channels, ASICs and HV DAC channels	52

Release notes

Version 2025.07

Changes to compilation instructions and requirements installation.

Version 2025.05

Support for real-time data processing while acquiring data. GUI update for new functionalities.

Version 2025.01

Support for new HW trigger and simulation of its performance using data processing applications.

Version 2024.08

New acquisition start options. Options for choosing the time reference for output timestamps.

Version 2023.07

New ASIC enabling mask. New compact output formats for processed data. Changes to GUI topology detection and bias calibration usage. General revision

Version 2022.04

Changes to the recommended OS. Support for time offset calibration. Documentation for `sw_trigger` parameters.

Version 2021.06

Revision of the overall structure of the guide.

Version 2021.03

Changes to the order of commands presented in chapter 3.

Version 2020.07

Support for changing the integration time of the QDC. Added extra plot to QDC calibration summary. Upgrade of the temperature monitoring.

Version 2019.09

Support for mixed mode (TOT and QDC) acquisition.

Version 2019.08

New energy calibration/linearisation introduced in software and documented. New version of “acquire_sipm_data”, allowing to acquire data while varying some key parameters.

Version 2019.01

New Graphical User Interface (GUI).

Version 2018.09

Updated software usage, new functions for added functionalities. New code example for multiple step data acquisition. New plot for QDC calibration summary.

Version 2018.07

Software revision, with new usage commands on several programs. Compatible with new BIAS-16P mezzanine. Requires new FEB/D firmware (v2018.06.29 or later).

Version 2018.05

Data acquisition with external gate signal.

Version 2018.04

Reference to temperature stabilization and usage of “convert_raw_to_group”

Version 2018.02

Major revision and new section regarding the usage of an external trigger.

Version 2017.11

Changed plots for calibration diagnosis and example CTR plot in Quickstart chapter to reflect new QDC calibration.

Version 2017.06

First version of Software package and Software User Guide for TOFPET2 ASIC.

Disclaimer

Information provided by PETsys Electronics SA is believed to be accurate and reliable. However, no responsibility is assumed by PETsys Electronics SA for its use, nor for any infringements of patents or other rights of third parties that may result from its use. Specifications subject to change without notice. No license is granted by implication or otherwise under any patent or patent rights of PETsys Electronics SA. Trademarks and registered trademarks are the property of their respective owners.

Chapter 1

System Overview

For Software downloads, future updates, and other related documents, please check PETsys Electronics documentation webpage (after registration):

<https://www.petsyselectronics.com/web/private/pages/18>

The following sections provide a brief overview of the hardware and software specifications.

1.1 TOFPET2 ASIC Evaluation Kit

As illustrated by figure 1.1, the system is composed of:

- Front end boards - type D (FEB/D) set, each being able to hold up to 8 TOFPET2 ASIC test boards. The FEB/D set is composed of FEB/D motherboard, a HVDAC module and a GBE Communication mezzanine.
- Two TOFPET2 ASIC Front end modules (FEM128) and cables.
- A computer, running 64-bit Linux.

The FEM128 are composed of 2 front end boards - type A (FEB/A, each holding one TOFPET2 ASIC, 1 front end board - type I (FEB/I) and 1 front end board - type S (FEB/S).

The FEB/D set provide the TOFPET2 ASIC test boards with power, bias voltage for the SiPM, a filtered low jitter clock (200 MHz, period of 5ns) and a FPGA interface, which implements the communication logic necessary to configure the ASICs, as well as reading out the data produced by the ASIC. It can also generate a test pulse, with an adjustable delay in relation to the ASIC reference clock.

Communication between the FEB/D FPGA and the computer is handled via Gigabit Ethernet.

1.1.1 Programming the FEB/D FPGA

Although the eKit is delivered with the firmware already installed, the user may need in the future to upload updated versions. Instructions to program the FEB/D FPGA can be found in the manual “PETsys TOFPET 2 ASIC SIPM Readout System - Firmware User Guide.pdf”. The firmware (.bin) files to be uploaded are provided by PETsys Electronics in the Client Package folder.

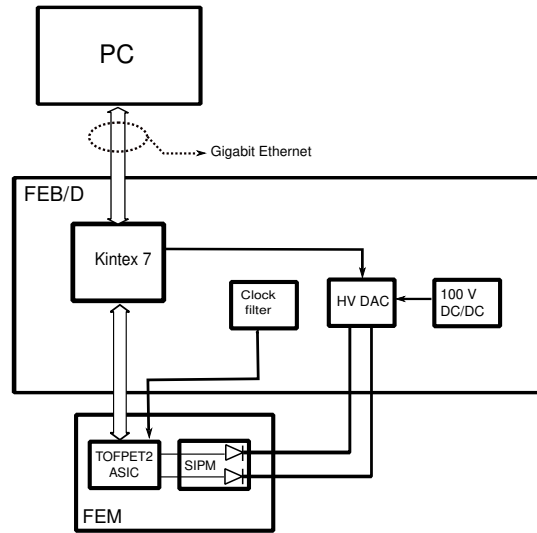


Figure 1.1: System Diagram - TOFPET2 ASIC Evaluation Kit

1.2 Computer requirements

The data acquisition computer must be capable of running 64-bit Linux and must have one available Gigabit Ethernet port. The Gigabit Ethernet interface only works at 1 Gbit/s, it is not capable of communicating with slower 10 or 100 Mbit/s ports.

This Ethernet port needs to be configured with the following parameters:

- IP address: 192.168.1.1
- Netmask: 255.255.255.0

The operative system must be 64-bit Linux. The distributions supported by PETsys Electronics are: CentOS Stream 8, CENTOS 7-2009, Red Hat Enterprise Linux 8.5 or above, and Ubuntu 20.04 and 22.04. The software includes a script to install all software dependencies and requirements for these distributions.

1.3 Software

The computer is used to run a suite of software in order to configure the system and process the data produced by the ASICs.

As shown in figure 1.2, the acquisition software consists of three main parts.

“daqd” is a communication server which must always be running in order to communicate with the hardware. Its main purposes are to abstract away the communication method and to act as a buffer to the data provided by the hardware.

The acquisition applications, communicating with the hardware through “daqd”, configure the hardware and acquire the data into files as needed.

Finally, processing applications are used to process and analyze the acquired data.

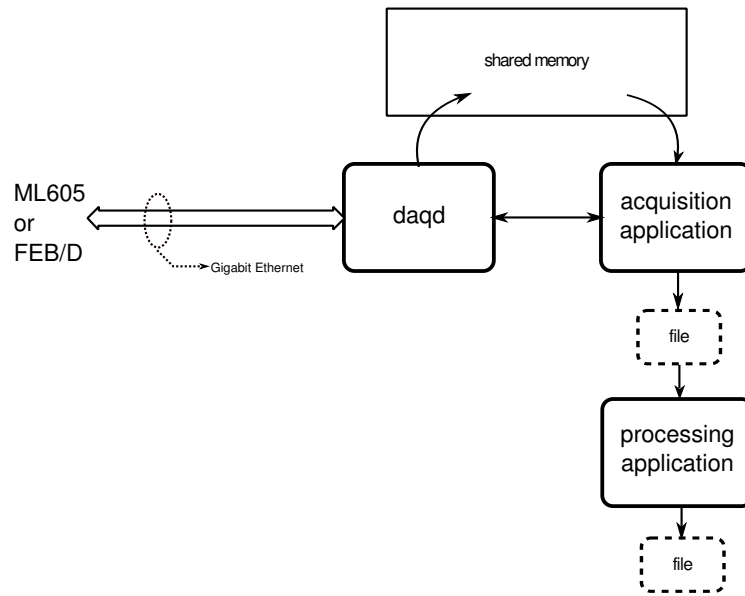


Figure 1.2: Software diagram

Communication between “daqd” and the acquisition applications is implemented through a UNIX socket (“/tmp/d.sock”) and a POSIX shared memory block (“/dev/shm/daqd_shm”).

All software applications consist of Python scripts (for communication with the hardware, configuration and data acquisition) or C++ compiled binary programs (for processing calibration/raw acquisition data) and can be executed directly from a UNIX/LINUX terminal (command line). Additionally, a graphical user interface (GUI) is available and can be used to run most of the software applications. The present guide illustrates how to use either command line or GUI for each required process.

1.4 Before first use

After downloading the latest software archive and copying the file to any desired folder, extract it by typing in a command line shell:

```
tar xvzf sw_daq_tofpet2-YYYY.MM.DD.tar.gz
```

This will extract the files into folder “sw_daq_tofpet2-yyyy.mm.dd”. Navigate to this newly created folder and create a “build” folder (to store all compiled applications):

```
mkdir build
```

Next, install all requirements by running:

```
su -
./petsys-setup
exit
```

If using CENTOS 7-2009, prepare the build compilation by typing (in the “build” folder):

```
cmake3 ..
```

If using CENTOS Stream 8 or above, RHEL 8.5 or above, Ubuntu 22.04 or above:

```
cmake ..
```

The final step is to compile the C++ applications and configure the Python scripts:

```
make
```

Chapter 2

Quickstart

The purpose of this chapter is to guide the user to quickly acquire data with your recently acquired hardware, process such data filtering for (time) coincidence events and plotting charge/energy spectra and coincidence time resolution (CTR). For a detailed description on each step please consult the remaining chapters of this user guide. For this Quickstart, the user must a point source isotope emitting 2 Gamma photons of 511 keV, typically a ^{22}Na point source with an activity higher than 10 μCi (370 kBq). The Gamma photons are to be detected by either a single SIPM pixel (provided by PETsys Electronics) or an array of SiPMs coupled to scintillation crystal(s) connected to the FEM boards. In order to measure Gamma events in coincidence, the ^{22}Na source should be placed in a virtual line between a pair (or several pairs in the case of two arrays) of crystal+SIPM pixels. The FEM boards should be connected in ports F1 and F3 of the FEB/D.

The user should make sure that all hardware is properly connected and powered. The Ethernet cable should be plugged and the PC Ethernet card configured as described in Chapter 1.

The setup should be placed inside a dark box to avoid any light contamination and preferably with active cooling.¹

2.1 Starting the GUI

In order to start the GUI, just open a system terminal, navigate to the software base folder and type:

```
./start_gui
```

This will open the GUI (please note that if the terminal from which it was launched is closed, the GUI will also close), which is shown in Fig. 2.1.

¹IMPORTANT NOTE: In order to avoid any damage to the hardware and ASICs, always power off the FEM boards (using the GUI FEM power switch or turning off the FEB/D) when plugging or unplugging any hardware and never perform data acquisitions with SIPMs under ambient light.

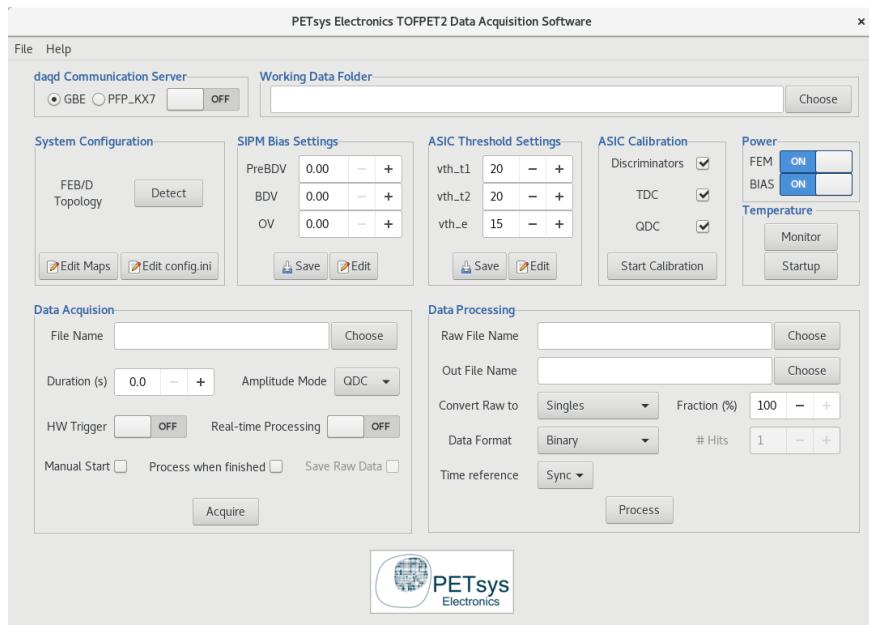
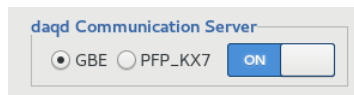


Figure 2.1: PETsys Electronics Data Acquisition Software: Graphical User Interface (GUI)

2.1.1 Launching “daqd”

Before any communication with the hardware is attempted, “daqd” must be running. Information about the type of connection used for communication is required. In the section “daqd communication server”, choose the option “GBE” and turn the switch on:



The system will open a new terminal automatically and, if a successful communication is established with the FPGA, the initial output on “daqd” will read (the size of frame may be different than this example):

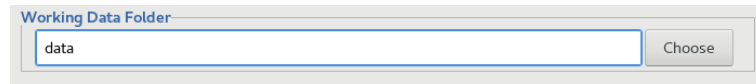
```
Got FPGA reply
Size of frame is 16384
```

Otherwise, the GUI will show an error message. If this happens, the user should check that all cables and hardware are properly connected and powered up and retry. Since “daqd” is the communication server, the terminal in which it is running should not be closed at any time. Any other output in the “daqd” terminal/window after this initial message should NOT be taken into account, since it is only for debug purposes. If an error occurs, a message will be shown either in the GUI itself or in the system terminal that was used to launch the GUI.

2.2 Configuration and calibration

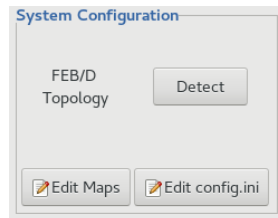
The principle behind the system acquisition data storage is that all calibration, configuration and acquisition data files are stored in the same folder, hereafter named Working Data Folder (see Chapter 3 for more details).

The first step should be to create this folder, which can be done using the GUI. In the section “Working data folder”, just type the folder name to be created (“data” in this example) :



and press Enter. Alternatively, the folder can be created using the “Choose” button, followed by “create folder” icon in the opened dialog. After confirmation, the folder is created and defined as the current Working Data Folder.

The next step consists in detecting the FEB/D set and save required configuration files into the Working Data Folder. This is done automatically in the GUI by using the “Detect” button in the section “System Configuration”.



After pressing the “Detect” button, a window with the description of the detected FEB/D and Bias Mezzanine type will show. After pressing “OK”, a new window will ask if the user wants to save this configuration. Press YES, and the interface will generate all the required map files, bias calibration files, configuration reference file and save them in the working data folder. For Evaluation Kits shipped before June 2018, the BIAS mezzanine is BIAS-64P (instead of “BIAS-16P” or “Bias-32P”) and in that case the user will be asked to enter the calibration file provided by PETsys Electronics before saving.

Before proceeding to acquire data, the user should check what is the recommended Breakdown Voltage and Overvoltage to bias the SIPMs that are connected. This information needs to be saved into a table to serve as reference for the software. PETSYS Electronics has provided its costumers with different types of individual 3x3mm SIPMs (coupled to 3x3x5mm LYSO crystals). The recommended settings are shown in Figs 2.2 and 2.3. If using other SIPMs, set the “BD” to the Breakdown voltage of your

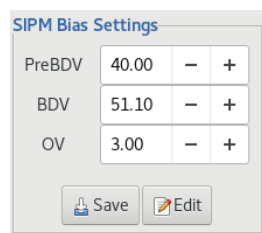


Figure 2.2: Bias Settings for Hamamatsu S13360-3050VE (Shipped before December 2024)

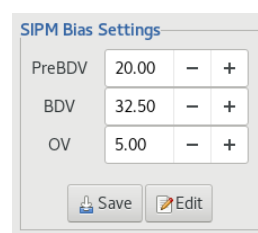
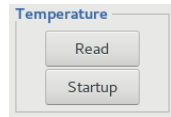


Figure 2.3: Bias Settings for Broadcom AFBR-S4N44P014M (Shipped after January 2025)

SIPM and “OV” to the desired working overvoltage. “PREBD” should be set to a value about 80 % of the

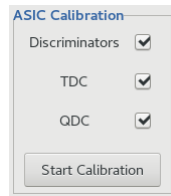
Breakdown Voltage. After setting the SIPM bias settings, these should be saved using the corresponding “Save” button on the bottom-left.

Some of the internal TOFPET2 ASIC circuitry, like the TDCs and QDCs, require calibration. However, before starting any calibration procedure, it is important to check for temperature stability of the ASICs (large temperature shifts can have an effect on TDC and QDC measured codes). In order to do so, just press the button “Startup” on the Section “Temperature”. This will open a new window and the



measured temperatures from all connected ASICs will be displayed every minute. When all measured temperatures are stable over one minute, the program will finish, indicating that the system is ready to acquire data. If no temperature stabilization is achieved in 10 minutes or more, the user must ensure that the external cooling system is properly installed and retry.

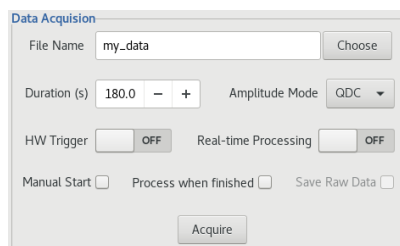
The user may then proceed to perform the calibration. After ensuring that all three options in the section “Calibration” are toggled, just press the button “Start Calibration”. A prompt window will open



requiring confirmation, and informing that this procedure can take some time to run. After confirming, a process window will open and run a suite of calibration scripts, taking more than 40 minutes to finish (the exact run time depends on the computer and the number of FEMs connected in the system). Configuration tables and plots (1 plot per ASIC) will be saved in the previously created folder “data”. During the calibration procedure the discriminator thresholds will be saved to the default values: $vth_t1 = 20$; $vth_t2 = 20$; $vth_e = 15$. This means that, for the purpose of this chapter, the user doesn’t need to perform any action on the section “ASIC Threshold Setting”. Nevertheless, the possibility of changing the ASIC discriminator thresholds is documented in Sec. 4.1 and can only be performed after finishing the calibration.

2.3 Data Acquisition with a radioactive ^{22}Na point source

The user is now ready to acquire data with the evaluation kit. To do so, type the name of the desired output file in the “Data Acquisition” section:



“Mode” should be set to “QDC”. The acquisition time depends on the activity of the ^{22}Na source, 180 seconds should be enough for a ^{22}Na source of about $20\ \mu\text{Ci}$. If using a lower activity source, then the acquisition time should be increased accordingly. “HW Trigger” can be set either “on” or “off”. After pressing “Acquire”, a process window will open printing the amount of data frames and time acquired. At the end of the acquisition, a small report will be printed containing the number of acquired data frames and the number of registered events. If the later number is 0, the user must make sure that the correct bias voltage is being set and that all boards, including SIPMs are tightly connected.

2.3.1 Coincidence events and Coincidence Time Resolution

If the ^{22}Na source was placed in between two SIPM detectors, one can process the data and filter it to select only coincidence events (events that correspond to the detection of two Gamma photons within a short time window and in different geometrical regions in the system). To do so, set the section “Processing data” as follows (the name of the input and output files are by default automatically suggested based on the raw data acquisition file name and final output event type, chosen from the drop down menu of “Convert Raw to”, and the final data format)

and press the button “Process”. A new window will open and after processing the raw data (which can take a few seconds), it will print a report in which the last printed number corresponds to the number of coincidence events found. The output file “data/my_data_coinc.ldat” will store a list (in this case, in binary format) with information on the channel ID, timestamp of arrival and energy (QDC measurement in arbitrary units) for each of the two events forming a coincidence. The user can then run the provided script “psDrawCTR.C” to plot the energy spectra on both channels as well as the coincidence time difference distributions. This should be done outside the GUI, by opening a system terminal, navigating to the software folder and typing:

```
root -l psDrawCTR.C+' ("data/my_data_coinc")'
```

An example of the output is presented in fig. 2.4. The script plots three histograms: two coincidence energy distributions, one for each of the two channels with the higher number of coincidence events. For each energy distribution, a gaussian fit is performed in order to determine the mean value and dispersion of the photopeak corresponding to the 511 keV gamma photons. In the middle panel, it is plotted the histogram for the time of arrival difference between the two photons for every coincidence event. To obtain this distribution, only events with energies compatible with a 1-sigma energy window around the photopeak centres are selected. The fit of a gaussian model to this distribution determines the time of flight variable and also the Coincidence Time Resolution (CTR) for that channel pair. In this example, the CTR is 85.7 ps sigma, which corresponds to 202 ps FWHM.

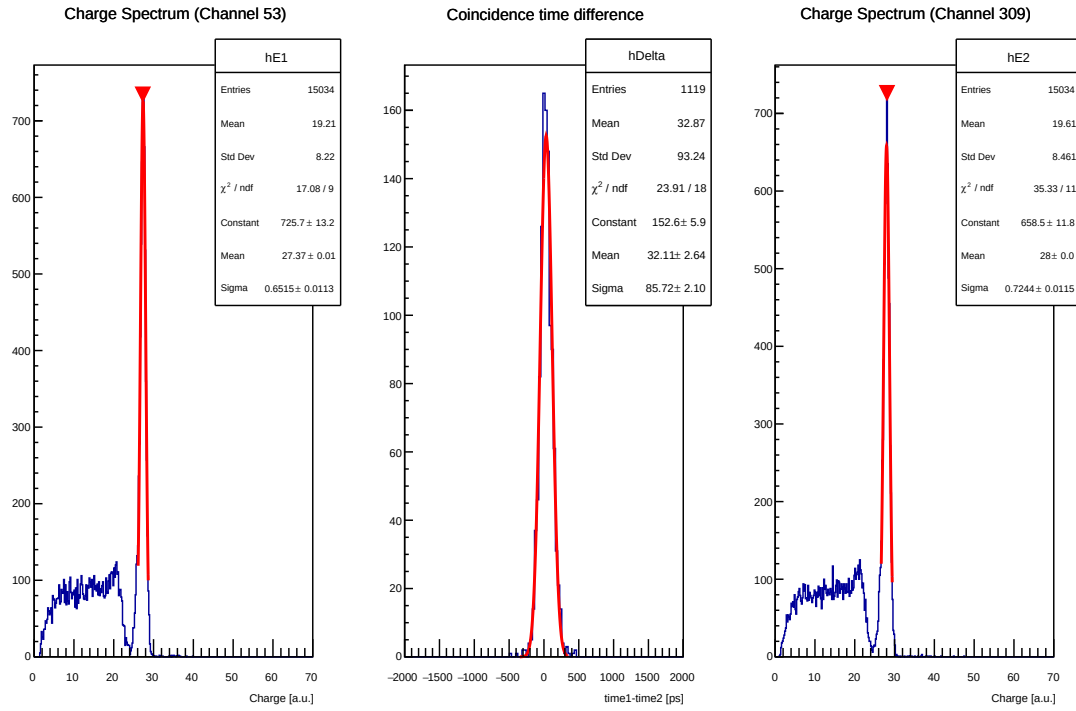


Figure 2.4: Example for the output of “psDrawCTR.C”, showing spectra (from QDC) and coincidence time resolution (CTR) for a channel pair reading two KETEK PM3325-WB SIPMs coupled to 3x3x5mm LYSO crystals.

Hopefully, the present chapter allowed the user to quickly obtain data and CTR results from your new Evaluation Kit. The next chapters will describe in more detail the routines and procedures of calibration, data acquisition and data processing.

Chapter 3

Reference Configuration File and Calibration procedure

NOTE: As previously mentioned, PETsys Electronics TOFPET2 data acquisition software is essentially based in Python scripts or C++ compiled executable applications, which can be called from any system terminal (command line approach). The available GUI manages to wrap almost all possible commands in a self-contained interface, and thus can be used standalone. Nevertheless, the user may still want to use the command line approach, thus we also document this in the next three chapters.

In order to operate properly, the system requires a set of different of configuration and calibration files, in the form of TAB separated tables. Configuration files are used to configure the ASICs and SIPM biasing for data acquisition and calibration files are used to obtain correct event time and charge/energy from the raw data transmitted by the ASICs. Within the software framework, all the required files for configuration and calibration are referenced in a single file: the Reference Configuration File - “config.ini”. This file is a required argument for almost all command line software applications, either for data acquisition or data processing.

A template of such file is provided in the software package and is show in listing 3.1.

CDIR is the data directory

Listing 3.1: config.ini

```
1 [main]
2 bias_calibration_table = %CDIR%/bias_calibration.tsv
3 bias_settings_table = %CDIR%/bias_settings.tsv
4
5 tdc_calibration_table = %CDIR%/tdc_calibration.tsv
6 qdc_calibration_table = %CDIR%/qdc_calibration.tsv
7 disc_calibration_table = %CDIR%/disc_calibration.tsv
8
9 acquisition_mode_table = %CDIR%/acquisition_mode_settings.tsv
10 disc_settings_table = %CDIR%/disc_settings.tsv
11
12 #energy_calibration_table = %CDIR%/energy_calibration.tsv
13 #time_offset_calibration_table = %CDIR%/time_offset_calibration.tsv
14
15 channel_map = %CDIR%/map_channel.tsv
16 trigger_map = %CDIR%/map_trigger.tsv
17
```

```

18 [hw_trigger]
19
20 hwtrigger_empirical_calibration_table = %CDIR%/
    hwtrigger_empirical_calibration.tsv
21
22 group_min_multiplicity = 1
23 group_max_multiplicity = 1024
24 group_min_energy = 0
25 group_max_energy = 128
26
27 pre_window = 2
28 post_window = 3
29
30 coincidence_window = 2
31
32 single_acceptance_period = 1000
33 single_acceptance_length = 0
34
35 [sw_trigger]
36 group_min_energy = -1e6
37 group_max_energy = +1e6
38 group_max_distance = 100.0
39 group_time_window = 20
40 coincidence_time_window = 2
41
42 [asic_parameters]
43 global.disc_lsb_T1 = 60

```

The file is divided in two sections. The “[main]” section (lines 2-12) defines the required configuration and calibration files, as well as map tables. These are:

- “bias_calibration_table” - Calibration table for the bias voltage DAC present on the FEB/D. Allows to provide correct bias voltage to SIPMs (up to 10 mV). The file is created with a software script from a previous calibration file provided by PETsys Electronics or from information saved in the EEPROM of the Bias Mezzanine board (systems shipped after June 2018).
- “bias_settings_table” - Configuration table containing 4 voltages (in Volts) to be defined for each Bias DAC channel of the Bias Mezzanine and applied to the SIPMs that are connected to the front-end modules when acquiring data. The 4 voltages are: “Offset” is an offset voltage to be added in order to compensate the voltage drop at the input of each ASIC channel. For TOFPET2 ASIC this value should be set to 0.75V; “Pre-breakdown”, a value below the breakdown voltage of the SIPM to be used by some calibration applications to acquire data with test pulses while having the SIPM capacitance as close to normal operation as possible; “Breakdown” is the breakdown voltage of the SIPM which serves as reference to set the “Overvoltage” to be applied in order to bias the SIPMs when acquiring data. It is defined by the user, and a script is provided to create this table, for the case where all channels provide the same value (see Section 3.1).
- “tdc_calibration_table” - Calibration table for the time-to-digital converters (TDC) present in each channel. Allows to correct the very low level of non-linearity in the correspondence between TDC codes and reference clock phase. Created by a calibration application provided by PETSYS Electronics (see section 3.2.2).

- “qdc_calibration_table” - Calibration table for the charge integrator (QDC) present in each channel. Created by a calibration application provided by PETSYS Electronics (see Section 3.2.3).
- “disc_calibration_table” - Calibration table for the 3 discriminators present in each TOFPET2 channel: t1, t2 and E. It contains the information for the baseline and noise level for each of the 3 discriminators, as well as dark count rate at different thresholds. Table obtained by running a calibration script provided with the software package (see Section 3.2.1).
- “disc_settings_table” - A configuration table that defines the threshold levels for each of the three discriminators. It is defined by the user, but a script is also provided to create this table in the case where all channels have the same value on a given discriminator (see Section 4.2).
- “channel_map” - A map table used by the post-processing applications to obtain information relative to a given system Channel ID. It can be defined by the user, since some of its elements depend on the system geometry, but a template with “dummy” values regarding SIPM physical geometry and pixel position can be created with a provided software function (see Section 5.4).
- “trigger_map” - A map table used by the hardware trigger filter and the software post-processing applications to identify the possible channel ID regions where to search for grouped events or coincidence events. A simple version is also created with the provided software function (see Section 5.4).

In the section “[hw_trigger]”, the user can change the in-built hardware trigger filter of events, as described in Sec. 4.2.3.

In the section “[sw_trigger]”, the user can change the processing software parameters for filtering of events, as described in Sec. 5.4.

The fourth section of the reference configuration file (“[asic_parameters]”) is reserved for changing the default values of the TOFPET2 ASIC configuration parameters. For typical PET applications with LYSO crystals, PETSYS Electronics only recommends to change t1 LSB parameter (“global.disc_lsb_T1”). It should be set accordingly to the gain of the SIPM and the pulse shape. According to the tests done by PETSYS Electronics, a proper working value is 57 (which corresponds to 5mV per DAC unit) for most SIPMs except for KETEK PM3325-WB, for which it can be set to 60 (corresponding to about 2.5mV per DAC unit). This choice implies that the amplitude for one photoelectron corresponds to about 5 DAC units of the “vth.t1” threshold.

If the user needs to change other ASIC configuration parameters, please refer to the document “PET-sys TOFPET 2 ASIC - Datasheet.pdf” where you can find information on parameter names and functionalities, as well as acceptable value ranges. The parameter values can be modified in this section, by defining the prefix “global.parameter_name” or “channel.parameter_name” for global ASIC parameters or channel specific parameters (in the later case all channels will be set to the same value).

The Reference Configuration File can be opened and edited directly from the GUI, with the button “Edit config.ini”. After performing any change, it is important to save the file using the same file name: “config.ini”.

3.1 Setting the working data folder, HV bias and mapping tables

Each row in the above mentioned files usually corresponds to a different ASIC channel (or channel TAC) or Bias DAC channel, depending on the file. The ID of such channels depends on which port of the FEB/D the TOFPET2 ASICs are connected (see Appendix A). This means that the configuration tables are valid if and only if they were obtained for a given configuration (connection of Front End Modules to FEB/D ports) and this configuration is not changed. It is IMPORTANT to create new configuration files

whenever this is not verified, or when using the hardware for the first time (as in the previous chapter of this guide).

All the files described above are to be located in the the same folder as the reference configuration file. Such folder (hereafter Working Data Folder - WDF) will also store the raw data files, as well as processed data containing relevant and calibrated event information. The first step then to create the WDF in the computer, using the OS or the GUI (as described in Sec. 2.2). Also, before running any script, please ensure that “daqd” server is running, either by using the GUI switch as described in Sec. 2.1.1 or by typing in a separate system terminal:

```
./daqd --socket-name=/tmp/d.sock --daq-type=GBE
```

Listing 3.2: configuration.template.sh

```
1 #!/usr/bin/sh
2 DATA_DIR=$1
3 CONFIG_FILE=${DATA_DIR}/config.ini
4
5 ./acquire_threshold_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}
   /disc_calibration
6 ./process_threshold_calibration --config ${CONFIG_FILE} -i ${
   DATA_DIR}/disc_calibration -o ${DATA_DIR}/disc_calibration.tsv --
   root-file ${DATA_DIR}/disc_calibration.root
7
8 ./make_simple_disc_settings_table --config ${CONFIG_FILE} --vth_t1 20
   --vth_t2 20 --vth_e 15 -o ${DATA_DIR}/disc_settings.tsv
9
10 ./acquire_tdc_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}/
   tdc_calibration
11 ./acquire_qdc_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}/
   qdc_calibration
12
13 ./process_tdc_calibration --config ${CONFIG_FILE} -i ${DATA_DIR}/
   tdc_calibration -o ${DATA_DIR}/tdc_calibration
14 ./process_qdc_calibration --config ${CONFIG_FILE} -i ${DATA_DIR}/
   qdc_calibration -o ${DATA_DIR}/qdc_calibration
```

The WDF should have a bias calibration file, which will be used to insure that correct voltages are provided by the Bias DAC channels. In order to create this file, type (assuming here that the WDF is named “data_folder”):

```
./make_bias_calibration_table -o data_folder/bias_calibration.tsv
```

This command will read the calibration voltages measured by PETsys Electronics and stored on each BIAS-16P mezzanine EEPROM and create a calibration file for the entire readout system, that can be used by the software programs.

If the user has an older Bias Mezzanine version (Bias-64P, shipped before June 2018), then the calibration file must be created from the files provided by PETsys Electronics, for example:

```
./make_bias_calibration_table -o data_folder/bias_calibration.tsv --port
0 --slave 0 --filename hvdacMezzXX_ad5535_cal.tsv
```

Also, the user should create a SIPM bias voltage table (“sipm_bias_table”) storing desired SIPMs bias voltage settings to be applied when acquiring data with the system. For example, if using KETEK PM3325-WB-D0 SIPM, the user can create a new table by typing:

```
./make_simple_bias_settings_table --config data_folder/config.ini
--offset 0.75 --prebd 20 --bd 24.9 --over 5 -o data_folder/bias_settings.tsv
```

In this example, the pre-breakdown voltage is being set to 20V, the SIPM breakdown voltage to 24.9V and the overvoltage to 5V. Due to a 750mV DC voltage at the input of each TOFPET2 ASIC channel, the offset voltage should always be set to 750mV (done automatically if using the GUI to save Bias Settings). Operational SIPM bias voltage in this example will be $24.9\text{V} + 5\text{V} = 29.9\text{V}$. This value will be set by the HV DAC in all its channels, whenever acquiring SIPM data. Pre-breakdown value is used when calibrating the ASIC. If the user needs to operate SIPMs at a different overvoltage, the “bias_settings.tsv” file should be manually edited to provide different values for the desired HV DAC channels.

3.1.1 Mapping tables

As described above, 2 different mapping tables are required by the software in order to provide correct coordinates of Gamma events. These should be created and saved in the working data folder whenever a new WDF is created. The 2 map files can be produced by a script, creating a simple version of the map, as described below.

Channel map table contents

The channel map table is a tab separated table text file, where each row has the information related to a given ASIC channel. The 10 columns are:

- portID: The DAQ port ID where the FEB/D is connected (see Appendix A).
- slaveID: The slave ID of the FEB/D when connecting multiple FEB/Ds in daisy chain (see Appendix A).
- chipID: The ASIC ID, related to the position where it is connected in the FEB/D (see Appendix A).
- channelID: The channel ID inside the ASIC (see Appendix A).
- regionID: Trigger region ID number (e.g., 0, 1, 2...).
- xi: Pixel index number in the X direction (0, 1, 2, ...).
- yi: Pixel index number in the Y direction (0, 1, 2, ...).
- x: Pixel spatial position in the X direction (in mm).
- y: Pixel spatial position in the Y direction (in mm).
- z: Pixel spatial position in the Z direction (in mm).

The trigger regionID allows to group ASIC channels into different regions (by attributing them the same trigger regionID). This definition will be used for selecting multi-hit groups and coincidence groups when processing the data, as described below for trigger map content.

The pixel index numbers in the X and Y direction are simply tagged to the event data when writing to the output ROOT data format and can be useful for data analysis. For example, these can be useful if the user wants to construct a 2D flood map of the SIPM array(s). The pixel spatial position values are used by the data processing software can also written to the output ROOT data format for analysis (see Chapter 4), allowing, for example, to obtain list-mode data to perform PET image reconstruction.

Trigger map table contents

The trigger map table has three columns, with the first 2 columns defining a trigger regionID pair and the third column defining the intended purpose of such trigger pair when processing the raw data: either coincidence (C) or multi-hit group (M). Typically, a multi-event (or multi-hit) group is composed of events occurring in the same trigger region, while a coincidence event requires two multi-hit groups to occur in different trigger regions (as described in section 5.3).

Creating a simple map

PETSYS Electronics provides a program to create a basic version of the maps mentioned above, grouping all channels connected to ports F1 and F2 (or F3 and F4, etc...) of the FEB/D into the same trigger regionID. In order to use this program, just type:

```
./make_simple_channel_map -o data_folder/map
```

This application will create both the channel and trigger maps. The last 5 columns of the channel map, corresponding to the geometrical position of the detectors connected to each channel, they can be filled by the user according to its system configuration or they can be left at 0. All data processing applications will still run correctly.

If using the GUI, the simple maps are created when using the “Detect” button in the “System configuration” section and after saving the configuration. They can be edited for customization with the “Edit Maps” button on the same section. Again, after editing, they should be saved using the same file names.

3.2 Generating ASIC calibration files

When both “bias_calibration_table” and “bias_settings_table”, as well as the channel and trigger maps are created and stored in the WDF, the user can proceed to create the required ASIC calibration files.

Generating new ASIC calibration files needs to be performed only whenever 1 of the following is true:

- Using the system for the first time.
- Changing the number of FEM/s in the system or changing the connection of FEM/s to the FEB/D.
- A temperature change of more than 3°C is observed in one or more ASICs (as measured by the ASIC temperature sensors), when compared to the temperature when the available/previous calibration files were obtained.

The script “configuration.template.sh” runs a series of applications that allow to create the three calibration tables described above (“disc_calibration_table”, “tdc_calibration_table” and “qdc_calibration_table”).

The ASIC(s) configuration script can then be run with the following command (assuming the user has renamed the script name from “configuration.template.sh” to “configuration.sh”):

```
sh configuration.sh data_folder
```

This is equivalent to using the “Start calibration” in the GUI, with the three options toggled, as described in Section 2.2.

3.2.1 Discriminator calibration

Lines 5 and 6 in listing 3.2 correspond to the data acquisition and processing for discriminator calibration. The data acquisition consists in a scan of the discriminator threshold in order to find the baseline and noise estimation (with the SIPM bias voltage set to a value below breakdown voltage). The script also performs a dark count scan, by performing the same threshold scan but this time with the SIPM biased at operational voltage. The data is processed in the last line which also allows to obtain the calibration table “data/disc_calibration.tsv”. Summary “svg” files, will be created for each ASIC, showing the values for the amplifier’s adjustment DAC, baseline and noise per channel. These should have data on all channels without a significant dispersion, as exemplified in Fig. 3.1.

3.2.2 TDC calibration

The 10th line in listing 3.2 launches a data acquisition where test pulses trigger the TDCs of all connected ASICs at different known phases relative to the reference clock. Line number 13 processes and fits the obtained data in order to determine the calibration function mapping measured TDC codes and time. It writes the parameters for those functions in the TDC calibration table, in this example the resulting file is “data/tdc_calibration.tsv”. The application will also create a series of ROOT files (containing all the histograms and data fits) and “svg” files, one for each ASIC connected. The summary plot for each ASIC should resemble the one on Fig. 3.2, with data on all 256 TACs (4 TAC per channel) for each discriminator and the time resolution after calibration on all of them below 0.01 clock period (RMS). If a significantly different plot is obtained for any ASIC, the user should contact PETSYS Electronics support team.

3.2.3 QDC calibration

Line number 11 of listing 3.2 acquires data to calibrate the charge integrator (QDC), by varying the integration window of test pulse driven events. The data is then processed in line number 14, in order to determine the offset current to be removed when integrating the charge of an event. A summary plot (Fig. 3.3) is created for each ASIC, showing the number of test pulse events used for the calibration and the measured resolution after calibration (difference between measured values by the QDC and the best-fit model). The measured resolutions should be under 4 ADC RMS, otherwise the user should contact the PETSYS Electronics support team. The summary plot also shows the minimum and maximum integration times, which can be changed by the user through the “imirror_bias.top” parameter, as described in section 5.7 of “PETsys TOFPET 2C ASIC - Datasheet”.

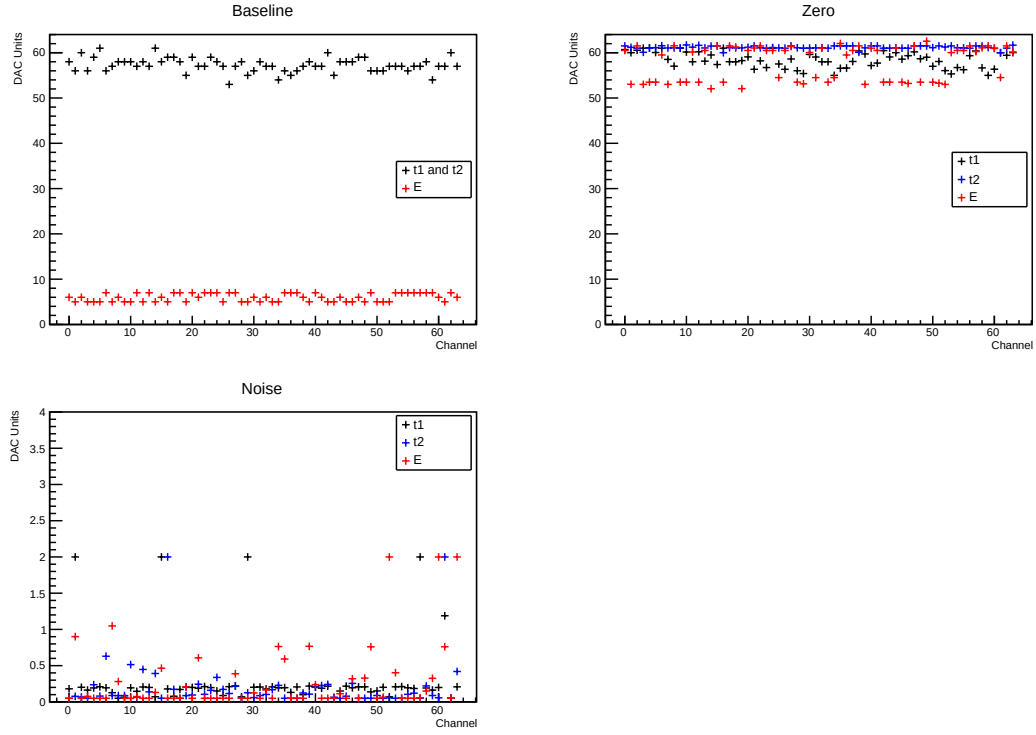


Figure 3.1: Discriminator calibration summary plots for one ASIC. On the top left plot, the value of the amplifier's adjustment DAC after convergence, per channel. On the top right plot, the value of the threshold level which corresponds to the amplifier's baseline, in DAC units, per channel. The bottom left shows the noise level measured per channel, again in DAC units. Different colors represent data from the 3 discriminators: t1 (black), t2 (blue) and E (red).

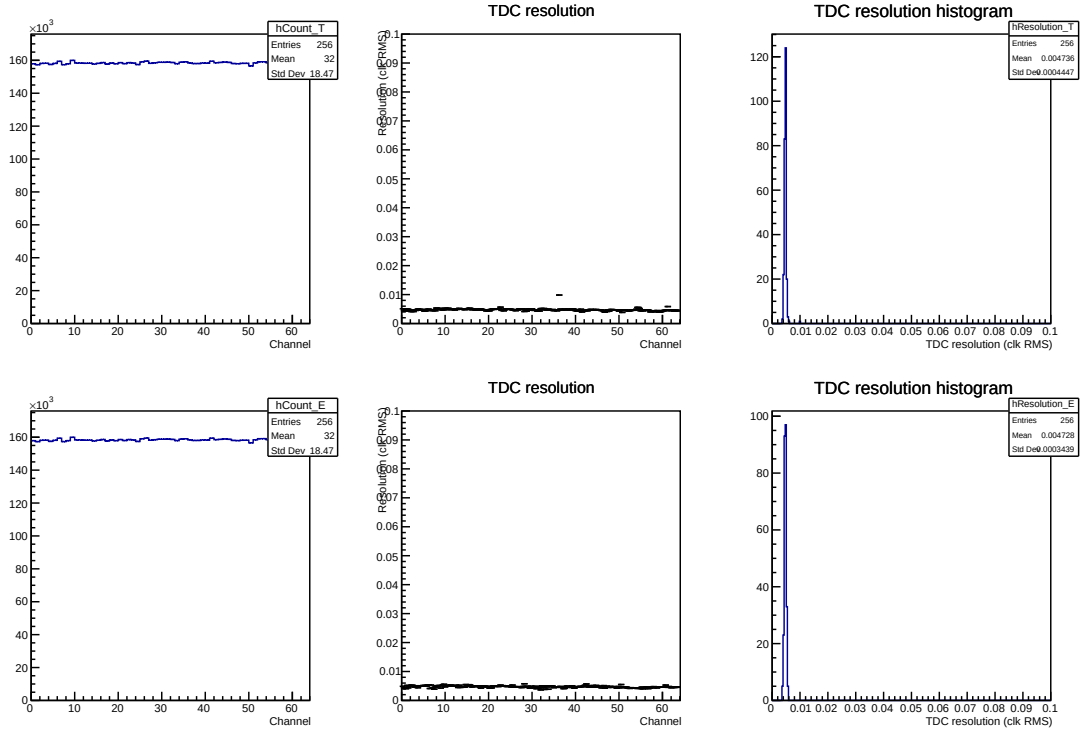


Figure 3.2: TDC calibration summary plots for one ASIC. The upper plots show the number of test pulse events used for calibration (left), the distribution per channel of the obtained TAC resolution after calibration (middle) and the overall distribution of resolutions obtained after calibration (right), for the first discriminator (t1 in default trigger mode). The plots on the bottom are the same but for the second discriminator (E in default trigger mode).

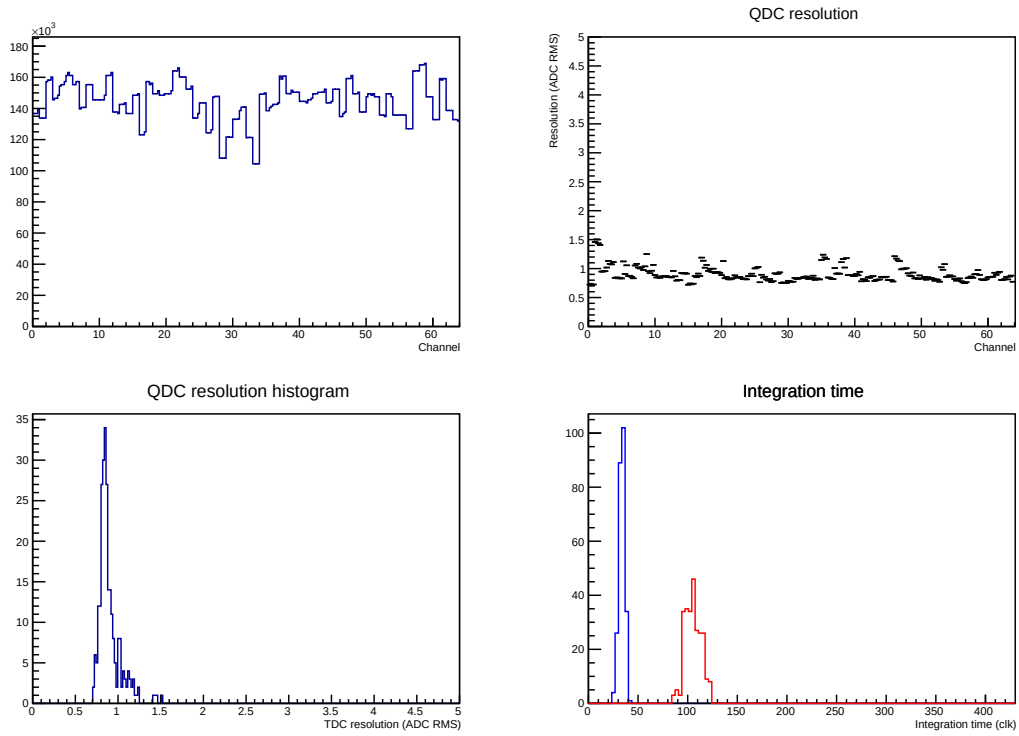


Figure 3.3: QDC calibration summary plots for one ASIC. On top left, the number of events used to calibrate each channel. On top right, the distribution for all channel and TACs of the average resolution after calibration. On bottom left, the histogram of the average resolutions for all channels of the ASIC. On bottom right, the minimum (blue) and maximum (red) integration times estimated by the software.

Chapter 4

Data acquisition

The acquisition application software consists mainly of a series of Python scripts and libraries which implement 3 main functions.

- Configure the hardware.
- Acquire raw data.
- Store raw data into files.

4.1 Data acquisition template

A basic data acquisition Python script is provided for reference and shown in listing 4.1. This Python program performs the following basic tasks (after parsing the arguments, which ends at line 14):

- Read and store configuration and calibration data from the files defined in the reference configuration file
- Connect to “daqd” and initialize the hardware.
- Load the hardware configurations and define operational bias voltage (from config file)
- Modify the ASIC configuration as needed (in this case setting the two configuration options related to TOT or QDC).
- Upload the ASIC configuration into the hardware.
- Open the output raw data file for writing.
- Acquire data during a period of time.
- Set the HV bias voltage to 0 on all channels and exit.

Listing 4.1: acquire_sipm_data_basic

```
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 from petsys import daqd, config
5 from copy import deepcopy
```

```

6 import argparse
7
8 parser = argparse.ArgumentParser(description='Acquire data for TDC calibration')
9 parser.add_argument("--config", type=str, required=True, help="Configuration file")
10 parser.add_argument("-o", type=str, dest="fileNamePrefix", required=True, help="Data filename prefix")
11 parser.add_argument("--time", type=float, required=True, help="Acquisition time (in seconds)")
12 parser.add_argument("--mode", type=str, required=True, choices=["tot", "qdc"], help="Acquisition mode (ToT or QDC)")
13 parser.add_argument("--enable-hw-trigger", dest="hwTrigger", action="store_true", help="Enable the hardware coincidence filter")
14 args = parser.parse_args()
15
16 systemConfig = config.ConfigFromFile(args.config)
17
18 daqd = daqd.Connection()
19 daqd.initializeSystem()
20 systemConfig.loadToHardware(daqd, bias_enable=config.APPLY_BIAS_ON, hw_trigger_enable=args.hwTrigger)
21
22 qdcMode = args.mode == "qdc"
23
24 asicsConfig = daqd.getAsicsConfig()
25 for ac in asicsConfig.values():
26     gc = ac.globalConfig
27     for cc in ac.channelConfig:
28         cc.setValue("trigger_mode_1", 0b00)
29
30         if not qdcMode:
31             cc.setValue("qdc_mode", 0)
32             cc.setValue("intg_en", 0)
33             cc.setValue("intg_signal_en", 0)
34
35         else:
36             cc.setValue("qdc_mode", 1)
37             cc.setValue("intg_en", 1)
38             cc.setValue("intg_signal_en", 1)
39
40 daqd.setAsicsConfig(asicsConfig)
41
42 daqd.openRawAcquisition(args.fileNamePrefix, qdcMode=qdcMode)
43 daqd.acquire(args.time, 0, 0);
44
45 systemConfig.loadToHardware(daqd, bias_enable=config.APPLY_BIAS_OFF)

```

4.2 Data acquisition procedure

Before acquiring any data, the user needs to define the threshold values for each of the ASIC's discriminators. These values are defined in a table which is referenced in the "disc_settings_table" field of the reference configuration file. For most SIPMs tested by PETSYS Electronics in the default triggering mode, we recommend starting with threshold of 20 DAC units above the baseline value for "vth_t1" discriminator, the same for "vth_t2", and a value of 15 DAC units above the baseline for the "vth_e" discriminator. Note that the value of "vth_t1" can be optimized to find the optimal timing resolution. The user can create a table for setting the same threshold value in all connected channels by running the "make_simple_disc_table" application. For the example of the recommended settings, this can be done by typing:

```
./make_simple_disc_settings_table --config data_folder/config.ini --vth_t1
20 --vth_t2 20 --vth_e 15 -o data_folder/disc_settings.tsv
```

The above command will write the output into the file named “disc_settings.tsv”, which is referenced in the configuration file. The user is free to choose other values for the thresholds, taking into account that the values in the table correspond to DAC units above the noise baseline of each channel and the DACs are 6-bit, ranging from 0 to 63.

If using the GUI, the above command is executed when using the “Save” button in the “ASIC Threshold Setting” section. The created file can be edited with the “Edit” button in the same section, for example, to set different threshold values to different channels. Here again, after editing, the file should be saved using the same file name.

With the threshold, SIPM bias and calibration tables defined and referenced in the configuration file, the user can acquire data from the SIPMs and ASICs. It is recommended to use the script “acquire_sipm_data”. The script can be ran as follows:

```
./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 60
-o data_folder/data_prefix
```

Each required (or optional) argument is preceded by a corresponding flag. The user may check all flags and argument descriptions by typing “-help” anywhere after “./acquire_sipm_data”.

In the above data acquisition example, the first flag is “-config” which is used to specify the reference configuration file described in chapter 3. Other required flag is “-mode”, which can only take three options: “qdc”, “tot” or “mixed”. In “qdc” mode, the output energy measurement for each pulse is obtained using the internal QDC for charge integration. PETSYS Electronics recommends to use this mode (for better resolution and linearity), except for very large signals that could saturate the integrator, or small amplitude signals (less than a few tens of photo-electrons). If using “tot” mode, the energy measurement is obtained from Time-over-threshold, using the difference between time stamps of E and t1 discriminators (non-linear measurement). The “mixed” mode option allows to acquire data with a part of the system’s channels acquiring in “qdc” mod and the other part in “tot” mode. In order to use this option, the user needs to manually create and save a table with the name “acquisition_mode_settings.tsv” in the working data folder, defining a mode for each of the system’s channels. An example of the format of such file is presented below:

```
#portID slaveID chipID channelID mode
0        0        0        0        qdc
0        0        0        1        tot
0        0        0        2        qdc
0        0        0        3        tot
...
0        0        1        0        tot
0        0        1        1        qdc
0        0        1        2        tot
0        0        1        3        qdc
...
```

All acquisition modes are selectable in the GUI within the “Mode” drop down list.

The “-time” flag sets the amount of time of the data acquisition, 60 seconds in the above example. If using the GUI, it can be set in the “Time” entry box.

The “-o” flag sets the output data file prefix. In this example, the script will produce 2 files named “data_prefix.rawf” and “data_prefix.idxf” and placed inside the folder “data_folder”. The file “data_prefix.rawf” contains the raw data in compact binary format. Its exact structure is not documented in this guide since the time information obtained directly from the ASICs, as well as the energy measurement from the integrator require correction using calibration on the post processing level, as previously described. The “data_prefix.idxf” file is an auxiliary text file containing information on the range of events for each of the variables “step1” and “step2”, which is only relevant if the user is performing parameter scans as described below.

If using the GUI, the acquisition is performed when pressing the “Acquire” button, and it will use the corresponding options as they are selected. If pressing the “Acquire & Process” button, the same applies, but immediately after finishing the raw data acquisition it processes it according to the options in section “Data Processing” (see chapter 5 for details on data processing).

4.2.1 Enabling only a subset of ASICs for acquisition

The system initialization sequence, called by “acquire_sipm_data”, has the option to enable only a subset of all the TOFPET2 ASICs connected to a FEB/D (the same subset will apply to all connected FEB/Ds). This option is implemented through a mask variable defined through a system environmental variable PETSYS_ASIC_MASK. The mask is an hexadecimal word where each binary significant bit corresponds to one chipID (1 to enable, 0 to disable). A script tool is provided to create the corresponding mask for any user selected list of asic IDs (see Appendix A). For example, if the user only wants to enable ASICs with chipID 0, 2 and 4, one can type:

```
./make_asic_mask --asics 0 2 4
```

The application “make_asic_mask” checks if the desired ASICs are connected in the system and provides the command to be typed by the user for exporting the PETSYS_ASIC_MASK environment variable corresponding to the selected ASIC IDs. The input to “make_asic_mask” can also be provided in the form of a text file containing a column with the desired ASIC IDs. In that case, the usage changes to:

```
./make_asic_mask --asics_file asics_mask.txt
```

where “asics_mask.txt” content would be:

```
0
2
4
```

After exporting the PETSYS_ASIC_MASK, all subsequent data acquisition and calibration runs will solely target the selected subset of ASICs.

To remove an existing mask, type:

```
unset PETSYS_ASIC_MASK
```

4.2.2 Parameter scan

The script “acquire_sipm_data” allows to acquire data continuously to the same output file while varying up to 2 key parameters (grid scan). This can be useful in order to help the user to determine an optimal performance configuration. The operational parameters available for scanning are:

- “OV” - the SIPM Bias Overvoltage.
- “vth.t1” - t1 discriminator threshold, in DAC units relative to the channel baseline.
- “vth.t2” - t2 discriminator threshold, in DAC units relative to the channel baseline.
- “vth.e” - e discriminator threshold, in DAC units relative to the channel baseline.
- “disc.lsb.t1” - t1 discriminator DAC LSB. Its value allows to change the dynamic range for the t1 discriminator DAC.

In order to perform a parameter scan, the user needs to activate the optional flag “--param-table” and provide a table containing 1 or 2 TAB separated columns, where the header (first line) contains the identification of the parameter(s) to be varied and the next lines contain the values to be scanned. For example, if the user wanted to obtain results with SIPM overvoltages of 2V, 2.5V, 3V, 3.5 and 4V, while also varying the “vth.t1” discriminator value for a set of possible values, it could type:

```
./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 60  
-o data_folder/data_scan_prefix --param-table param_table.txt
```

Where the content of the file “param_table.txt” would be:

Listing 4.2: param_table.txt example

OV	vth.t1
2	10
2.5	20
3	30
3.5	40
4	50
	60

In this example, the script would then acquire for 60 seconds for each of the 30 possible parameter combinations, while writing the output raw data into the same file “data_scan_prefix.rawf”. The “data_scan_prefix.idx” would contain 30 lines with the information of the first and last event corresponding to a given parameter combination. Naturally, it is possible to define only one column for a single parameter scan. For the moment, the parameter scan feature is not implemented in the GUI, only in command line.

4.2.3 Hardware coincidence trigger

Although not used in the above example, the acquisition script has an additional optional flag “--enable-hw-trigger”. This will activate the usage of a firmware block (running on FEB/D and Clock & Trigger FPGAs) that can discard events during the acquisition based on (coarse) estimations of the time of arrival, channel IDs and corresponding trigger regions, but also on grouping of energy and multiplicity. This filter should be enabled whenever the user desires to reduce the amount of data transmitted to disk while keeping relevant data coincidence events, such as in a typical PET scanner application.

4.2.4 Multi-hit group and energy sum filter

As mentioned above, the coincidence trigger implements an algorithm to identify groups of events based on coarse time (after applying a first-order correction to account for possible time-walk effects) and computes a raw estimation of the energy sum (total deposited charge) of multiple events detected in different channels. Such feature is particularly useful for applications using light sharing detector modules, where the scintillation light generated by a Gamma photon creates pulses in several SIPM pixels and thus a multiple series of TOFPET2 events in different channels. A global minimum and maximum threshold values can then be set to control data acceptance or rejection, namely:

- “group_min_multiplicity” - The minimum group multiplicity accepted for transmission. Corresponds to the number of events occurring within the same trigger region and within a coarse time window of 20ns, after applying a first order time walk correction. Setting this value to 1 allows to transmit all events without filtering for minimum multiplicity.
- “group_max_multiplicity” - The maximum group multiplicity accepted for transmission. Corresponds to the number of events occurring within the same trigger region and within a coarse time window of 20ns, after applying a first order time walk correction. Setting this value to 1024 allows to transmit all events without filtering for maximum multiplicity.
- “group_min_energy” - The minimum group energy value to be accepted for transmission. Corresponds to the sum of all measured charge (in arbitrary units) for a group of events occurring within the same trigger region and within a coarse time window of 20ns, after applying a first order time walk correction. Setting this value to 0 allows to transmit all events without filtering for minimum energy.
- “group_max_energy” - The maximum group energy value to be accepted for transmission. Corresponds to the sum of all measured charge (in arbitrary units) for a group of events occurring within the same trigger region and within a coarse time window of 20ns, after applying a first order time walk correction. Maximum possible value is 128.

The firmware computation of energy and time walk requires calibration parameters uploaded to the FPGA memory as a look-up-table (LUT). Such constants should be obtained directly from the system data, originating in the detector modules (user and application specific). In order to do so, the user should acquire data without the coincidence filter activated (the duration of the acquisition depends on the activity of source, and detection modules geometry) and process it with the corresponding software module:

```
./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 120  
-o data_folder/data_for_hwtrigger_calibration_prefix
```

and process it with the corresponding software module:

```
./process_hwtrigger_empirical_calibration --config data_folder/config.ini  
-i data_folder/data_for_hwtrigger_calibration_prefix  
-o data_folder/hwtrigger_empirical_calibration.tsv
```

If the acquired data is insufficient for a good quality calibration, the program will report it. It also indicates for how many channels and TACs a successful calibration was obtained.

Please note that the calibration table is only required if the user wants to set the above threshold values to settings different from the default. For the default values, no filtering is applied for either energy (`group_min_energy = 0`; `group_max_energy = 128`) and multiplicity (`group_min_multiplicity = 1`; `group_max_multiplicity = 1024`). This means that for these settings, the hw trigger block operates only as a coincidence filter (see below), hence removing the need for the calibration table to be defined and present in the working data folder.

4.2.5 Coincidence filtering

In addition to the possibility of filtering individual groups of events based on energy sum and multiplicity, the hw_trigger firmware also discards non-coincident events based on their coarse (5ns resolution) arrival time and corresponding coincidence trigger regions (assigned to individual channel Ids). In the firmware implementation, the minimum trigger region definition is fixed and cannot be changed, which means that any changes made by the user to the channel map (described in 3.1.1) are disregarded. However, the trigger map table (section 3.1.1) is passed to the hardware trigger filter and can be used to select which trigger regions are allowed for coincidence events and thus transmitting data.

Besides the trigger map, there is a set of parameters available for adjusting the hardware trigger performance, which can be set in the “[hw_trigger]” section of the Reference Configuration File (“config.ini”). These are:

- “pre_window” - If a coincidence event is found, this variable sets the pre-event time window of accepted events, in clock period units (from 0 to 3).
- “post_window” - If a coincidence event is found, this variable sets the post-event time window of accepted events, in clock period units (from 0 to 15).
- “coincidence_window” - This variable sets the coincidence time window, in clock period units (from 0 to 7).

The hardware trigger also includes an inbuilt functionality allowing to transmit all events (suspend the filtering of non-coincidence events) for a configurable fraction of time during the acquisition. This option can be helpful when the user wants to estimate the single event fraction obtained with the hardware trigger enabled. This feature is controlled by two parameters (also in “[hw_trigger]” section):

- “single_acceptance_period” - Defines a cycling period of time over which to select a fraction of events. Units are in clock periods (from 0 to 65535).
- “single_acceptance_length” - Defines during how many clock periods, on each period of “single_acceptance_period”, the totality of events is transmitted. Any value larger than 0 activates the functionality. Units are in clock periods (from 0 to 65535).

For example, if “single_acceptance_period” = 1000 and “single_acceptance_length” = 2, unfiltered data will be transmitted for 2 clock periods every 1000 clock periods.

If using the GUI, the hardware coincidence filter can be toggled “on” or “off” in the “HW Trigger” option. The hardware trigger parameters can be changed from the GUI by using the “Edit config.ini” button, changing the values and saving the resulting file without changing its name.

4.2.6 Software simulation

It is possible to simulate with software data processing applications the same filtering algorithm implemented in firmware, thus allowing the user to test the efficiency of such filter on their data while optimizing the threshold values and coincidence acceptance parameters described above. The usage of the software simulation is documented in section 5.6 of the present guide.

4.3 Manual or internally triggered acquisition start

In all the examples shown previously, the acquisition script “acquire_sipm_data” sequentially prepares the system, synchronizes it and immediately starts transferring the data to the raw output file. Optionally, the exact moment for starting the data transfer into the raw data file can be activated by an external trigger or manually by the user. Basically, if the user launches the acquisition script using the flag “--wait-on”, followed by the path to a previously created “fifo”, the acquisition script will prepare the acquisition, synchronize the system, but then halt and wait for one byte to be written into a fifo to create and start filling the raw data file. For example, one can create a fifo by running:

```
mkfifo data_folder/start_fifo
```

followed by:

```
./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 60  
-o data_folder/data_prefix --wait-on data_folder/start_fifo
```

The program will then prepare the acquisition and wait for any byte to be written to “data_folder/start_fifo” in order to initiate the data acquisition. Please note that the fifo can be created and called for using any path in the system, there is no requirement to create it in the working data folder. This feature is also implemented in the GUI, providing an option for manual start by pressing a Start button. The user just needs to select “Yes” in the section that refers a Manual Start. This will minimize the time interval between the start of the acquisition and the first event, allowing the timestamp for the first event to be close to 0 (see section 5.5).

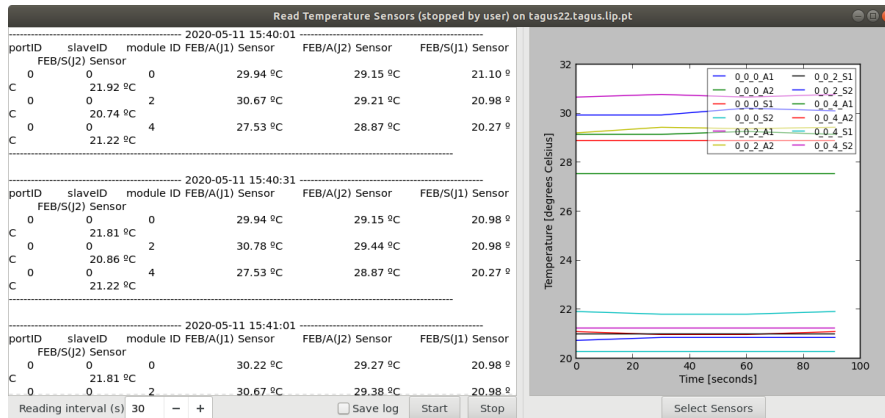
4.4 Temperature reading and stability

In order to read the temperature of your boards, the user can type in command line:

```
./read_temperature_sensors
```

This will print a report with the values of all the measured temperatures, grouping the 4 sensors (two per FEB/A board and two per FEB/S board) on each Front-end module plugged in the system. The user can also launch the script in order to monitor the temperature for a given time period reading the values at fixed time intervals. For that purpose, just use the optional flags “--time” and “--interval” with the corresponding desired values. When monitoring the temperature over time, the user also has the option to store the measured files in a log file. This can be done by using flag “-o” with the corresponding file name.

The temperature reading and monitoring can also be executed in the GUI, with additional displaying features. In order to do so, just click the “Monitor” button in the “Temperature” section. This will open a new window, in which the user can chose the time interval of temperature measurements and chose to register the values in a log file (by checking the appropriate box). After clicking “Start”, the measured temperature values will be print to the window and displayed in the plot on the right. The user may then chose to plot only a set of sensors, by clicking the “Select Sensors” button and un-checking some of the sensors. The ID of each sensor is made using the format “portID_slaveID_moduleID_sensorID”, where “portID” and “slaveID” are equal to 0 in the case of an evaluation kit. “moduleID” corresponds to the port in the FEB/D where the FEM is connected (0 to 7) and S1, S2 refer to the two sensors on the FEB/S while A1 and A2 to the two sensors close to the ASICs.



IMPORTANT: After any power cycle of the FEB/D board, the first run of the acquisition script will configure the ASICs (for the first time), causing a rise in temperature that can affect the measured time and energy stamps. In order to avoid this effect, we recommend running the following program (in the GUI, use the “Startup” Button in “Temperature” section) before the first data acquisition after a FEB/D power cycle:

```
./read_temperature_sensors --startup
```

The script will configure the ASICs with a default configuration, and measure the temperature from the sensors closest to the ASICs every minute. When all temperature are stable (up to 0.25°C) for 1 minute, the program will exit, indicating that the system is ready to acquire data. If no temperature stabilization is achieved in 10 minutes or more, the user must verify the air flow or cooling system and retry.

4.5 Other utility applications

As set of applications designed for operational utility is available in the software package. For instance, in order to check the system status at any time, the user may run:

```
./init_system
```

This program will print on screen the detected hardware and its IDs: (port, slave) for the FEB/D and ASIC ID

Other function that can be useful for the user, allowing to power off/on the Front End Modules is:

```
./set_fem_power --power off
./set_fem_power --power on
```

With the argument “off”, this program will power off all the FEMs in the system, including the Bias voltage for the SIPMs, thus allowing to safely remove or plug any detector or additional FEM, or even just turn on the lights safely.

This function can be applied with a corresponding switch in the “Power” section of the GUI, as well as a similar function that only turns “on” and “off” the bias voltage to the SIPMs and not the full FEM power.

4.6 Using an external trigger

It is possible for the TOFPET2 Evaluation Kit to accept signals for triggering. More information on this feature and how to set it in Software can be found in the document “Application note: Synchronization with external system.pdf”.

Chapter 5

Data processing

The acquired data saved on the disk consists of the measured values obtained from the ASIC TDCs, represented in ADC units. Converting these values into time is not a standardized process, although it closely resembles a linear relationship. To achieve such conversion, a calibration function specific to each ASIC channel is required (refer to section 3.2.2). Additionally, if the acquisition was performed in 'QDC' mode, further post-processing correction of the raw data provided by the ASIC is necessary (refer to section 3.2.3).

To use the calibration files and accurately convert the raw data into information containing precise time and charge measurements, the processing applications provided by PETsys Electronics must be employed.

5.1 Singles application

The main processing application is “convert_raw_to_singles”. This program provides a list of all registered events using minimal filtering (only excludes events where the measured time in ADCs is outside the operational nominal range).

In order to process data with this application (following the example from chapter 4), just type:

```
./convert_raw_to_singles --config data_folder/config.ini  
-i data_folder/data_prefix -o data_folder/data_singles.txt
```

In the above example, “data_folder/data_prefix” is the input (flag “-i”) raw data file prefix and “data_folder/data_singles.txt” is the output (flag “-o”) file.

The user has the flexibility to select the desired output data format from a range of available options. The default option (as in the example above) corresponds to “text” format. In this case, each line of the output file corresponds to one event and each of 3 the columns to the following quantities:

- Time of detection (in picoseconds).
- Energy of the pulse (for “QDC” mode, in arbitrary units of charge; for “TOT” mode, in nanoseconds).
- Absolute channel ID (see appendix A)

An example of the output of this application, “data_singles.txt” could be:

```

...
746540587230 1.391052 114
746540590891 1.612877 66
746540587241 4.427330 72
746540584157 14.772469 68
746540584648 6.921204 70
746540589927 1.254074 123
746540588194 2.033409 74
746540585346 4.360729 67
746540593966 1.428196 77
746540679252 17.219238 280
746540682757 1.993874 275
746540683588 4.370556 281
746540680081 13.847786 119
...

```

Another option is to write the output file in a compact binary format, which can be selected by using the “--writeBinary” flag when calling the application. If this option is set, the same three variables per event will be written to the output file in a format of 64 bit integer (time in picoseconds), 32 bit floating point (energy or ToT, depending on the acquisition mode) and 32 bit integer (channel ID). The C code structure for each event is:

```

1 struct Event {
2     long long time;
3     float e;
4     int id;
5 };

```

If using the binary output option, the output file defined after the “-o” flag is indeed a prefix, since two files will be created. The user should not define an extension for the file, and can run the application as:

```

./convert_raw_to_singles --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_singles --writeBinary

```

Two files will be created in “data_folder”: a “data_singles.ldat” file containing the binary data with the structure presented above and a “data_singles.lidx” file which contains information of the event range corresponding to different parameters acquired during a parameter scan as described in section 4.2.2. If the data acquisition did not involve a parameter scan, then can be discarded although it will still be created (containing a single line).

A third output data type is available with the application “convert_raw_to_singles”, consisting of a ROOT file with the data in the form of a ROOT TTree named “data” with one event per entry. This ROOT TTree contains more information besides the three entries described above, making this format the largest in terms of file size. The full TTree structure is:

- “step1” and “step2” are the values of step1 and step2 from the acquisition with multiple parameters. If no scan is performed, both this variables are set to 0.
- “time” is the time of the event, in picoseconds.

- “channelID” is the absolute channel ID of the event (see appendix A).
- “tac” is the TAC number for this event, ranging from 0 to 3.
- “tot” is the Time-over-Threshold for the event, in nanoseconds. In QDC acquisition mode, this time corresponds to the time of integration, while in TOT mode it is the difference between the time stamps at the E and t1 discriminators.
- “energy” is the energy of the event (in QDC mode, in ADC units; in TOT mode, in nanoseconds).
- “channelIdleTime” is the time between the event and the previous event in the same channel.
- “tacIdleTime” is the time between the event and the previous event in the same TAC.
- “xi” and “yi” are the index position in the X and Y directions of the pixel connected to channelID (obtained from the channel map file described in Sec. 3.1.1).
- “x”, “y” and “z” are the physical coordinates of the pixel connected to channelID (obtained from the channel map file described in Sec. 3.1.1).
- “tqT” is the fine time (time for crossing the time threshold) measured directly by the TDC in units of clock periods.
- “tqE” is the fine time (time for crossing the energy threshold) measured directly by the TDC in units of clock periods.

For ROOT output format, use the option “--writeRoot”, so the application should be run as:

```
./convert_raw_to_singles --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_singles.root --writeRoot
```

Option “--help” prints to screen all the arguments and flags for “convert_raw_to_singles”.

5.2 Group application

As mentioned above, the output data generated with “convert_raw_to_singles” contains all the events validated by TOFPET2 ASICs in the system, and the user can create any suitable data analysis tool to filter and extract additional information. However, PETsys Electronics software includes two additional data processing applications that provide extra features for grouping events based on channelID, timestamp and charge. These can be useful in the context of a typical PET application using pixelated or monolithic crystals coupled to SIPM pixel arrays.

The first of these programs is “convert_raw_to_group”. It is very similar to the singles application described in the previous section, in the sense that processes and retrieves time and energy information for all events applying minimal filtering, the difference being that it also identifies groups of events based on their time of arrival and detection position. Such groups can help to identify one single Gamma photon interaction in a crystal block or array when the scintillation light is detected in multiple channels. The program will consider any two (or more) events to be part of the same group if they are measured with a timestamp difference smaller than a given selectable time window (100 ns by default) and occur in SIPM pixels separated by a physical distance smaller than a given value (distances between channels are defined in the channel map, see sec. 5.4). Moreover, the two channels must correspond to the same trigger region (also defined in the channel map table) and the trigger regions that are expected to have grouped events (multi-hit) should be defined in the trigger map table (sec. 5.4).

In order to use the group application, type:


```
./convert_raw_to_group --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_group.txt
--writeMultipleHits 64
```

The option “--writeMultipleHits” defines the maximum number of events per group that will be written to the output file, in this example 64.

The default output file will have a text format. Each line corresponds to one event. As described above, in addition to the same information delivered by the singles application, two more integer numbers are provided for each event. The 5 columns correspond to:

- “mh_n”, the number of events that belong to the same group of this particular event. If “mh_n” is 1, then no other events arrived within 100 mm and within a 100 ns time window (default values).
- “mh_j”, the number ID (from 0 to “mh_n” - 1) of this event in the group it belongs to (ordered from higher to lower energy). For example, an event with “mh_j” set to 1 means this event has the second highest energy in the group it belongs to. By default this number is always 0, unless the option “--writeMultipleHits” is defined.
- Time of detection in picoseconds (for the event in the trigger region of higher ID number).
- Energy of the pulse, for the event in the trigger region of higher ID number (in QDC mode, in arbitrary units of charge ; in TOT mode, in nanoseconds). If the energy calibration file (see section 5.8) exists and is referenced in the reference configuration file, the energy value will also have arbitrary units of charge.
- Absolute channel ID, for the event in the trigger region of higher ID number (see appendix A).

An example of the output data stored in “data_folder/data_group.txt”:

```
...
13 0 746544972280 12.573620 94
13 1 746544974401 3.998756 90
13 2 746544976888 2.966228 93
13 3 746544974937 2.549541 100
13 4 746544975858 2.487907 95
13 5 746544990825 1.420338 98
13 6 746544980917 1.161053 88
13 7 746544985430 0.659710 92
13 8 746544974718 0.571350 75
13 9 746544979109 0.508190 103
13 10 746544983920 0.259937 96
13 11 746544982585 0.181274 97
13 12 746545017441 0.137691 105
7 0 746546240112 8.429253 79
7 1 746546242364 2.906200 66
7 2 746546242511 2.823254 72
7 3 746546243766 2.694466 70
7 4 746546245394 1.887131 68
7 5 746546243114 1.364708 123
7 6 746546276007 0.162373 114
```

```

1 0 746547881873 1.297798 310
8 0 746555808476 9.199627 275
8 1 746555812228 1.431965 273
8 2 746555838748 1.253864 279
8 3 746555813751 1.213791 281
8 4 746555818542 1.046593 278
8 5 746555838681 0.488628 280
8 6 746555813239 0.272110 272
8 7 746555825509 0.176234 284
...

```

Again, when using the option “-writeBinary”, the output file will have a compact binary format, with 5 variables per event structure and the data format will be: 8-bit unsigned integer (mh_n), 8-bit unsigned integer (mh_j), 64-bit integer (time in picoseconds), 32-bit floating point (ToT or Energy) and 32-bit integer (channel ID). The full C code structure for each event is:

```

1 struct Event {
2     uint8_t mh_n;
3     uint8_t mh_j;
4     long long time;
5     float e;
6     int id;
7 };

```

A third output data type is possible with the application “convert_raw_to_group” (using the flag “-writeRoot”), consisting of a ROOT format file, containing the data in the form of a ROOT TTree named “data” with one event per entry. There will be one set of “time”, “channelID”, “tac”, “tot”, “energy”, “mh_n”, “mh_j”, “xi”, “yi”, “x”, “y”, “z”, as branches of the TTree “data”.

5.2.1 Compact data formats

With the release of software version v2023.07.24, two additional output data formats were introduced for the application “convert_raw_to_group”, with the goal of increasing the compactness of the output data structure and hence reduce the size of processed data files. The three original data formats described above were kept available for compatibility with older reading programs.

The new formats include new output structure for text and binary forms, and can be selected by using the flags “-writeTextCompact” and “-writeBinaryCompact”. The first will write a text file where a header indicating the value of “mh_n” (the number of events in the group) followed by time, charge, and channel information for those grouped events. Using as example the data presented above, it changes in the new format to:

```

...
13
746544972280 12.57362 94
746544974401 3.998756 90
746544976888 2.966228 93
746544974937 2.549541 100
746544975858 2.487907 95
746544990825 1.420338 98
746544980917 1.161053 88

```

```

746544985430 0.65971 92
746544974718 0.57135 75
746544979109 0.50819 103
746544983920 0.259937 96
746544982585 0.181274 97
746545017441 0.137691 105
7
746546240112 8.429253 79
746546242364 2.9062 66
746546242511 2.823254 72
746546243766 2.694466 70
746546245394 1.887131 68
746546243114 1.364708 123
746546276007 0.162373 114
746547881873 1.297798 310
8
746555808476 9.199627 275
746555812228 1.431965 273
746555838748 1.253864 279
746555813751 1.213791 281
746555818542 1.046593 278
746555838681 0.488628 280
746555813239 0.27211 272
746555825509 0.176234 284
...

```

As for the binary compact form, it uses the same principle where the header for each group is a 8-bit unsigned integer (uint8_t). The events for each group are sequentially written after the header using the same C code structure used for "single" events:

```

1 struct Event {
2     long long time;
3     float e;
4     int id;
5 };

```

An example of a C code snippet for reading such format is:

```

1 #include <stdio.h>
2
3 struct Event {
4     long long time;
5     float e;
6     int id;
7 } __attribute__((packed));
8
9 typedef uint8_t GroupHeader;
10 Event ev;
11 GroupHeader nHits;
12
13 FILE * dataFile = fopen("data_folder/data_group.ldat", "rb");
14

```

```

15 while(!feof(dataFile)) {
16     fread(&nHits, sizeof(nHits), 1, dataFile);
17     Event group[nHits];
18     fread(group, sizeof(Event), nHits, dataFile);
19 }

```

5.3 Coincidence application

The last application provided in the software to filter and group events is "convert_raw_to_coincidence" which extends the filtering and grouping capabilities of the previous "convert_raw_to_group" application, allowing for the identification of multi-hit event groups that satisfy the "coincidence" criteria based on timestamp difference and trigger region information. In order to determine if two multi-event groups are in "coincidence", the application considers the following conditions:

- **Timestamp Difference:** The timestamp difference between the two groups must be smaller than a selectable value. By default, this value is set to 10 nanoseconds. This criterion ensures that the events in the groups are close enough in time to be considered coincident.
- **Trigger Region:** The two groups must originate from different trigger regions. The trigger region to which a particular channel belongs is defined in the channel map file. This criterion ensures that the events from different regions are considered for coincidence validation.

The required trigger region pairs for coincidence validation are defined in the trigger table map. This map provides the necessary information to determine which trigger regions should be compared for coincidence.

In order to use the coincidence application, just type:

```

./convert_raw_to_coincidence --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_coincidences.txt

```

The default output file will be also a text file where each line corresponds to two events from each of the groups in coincidence. As described above, one coincidence event consists of two grouped events, occurring in different trigger regions. The ten columns correspond to the following variables:

- "mh_n1", the number of events that belong to the same group of this particular event (for the event in the trigger region of higher ID number). If "mh_n1" is 1, then no other events were detected within a radius of 100mm and a 100 ns time window (default values).
- "mh_j1", the number ID (from 0 to "mh_n1" - 1) of this event in the group it belongs to (ordered from higher to lower energy, for the event in the trigger region of higher ID number). For example, an event with "mh_j1" set to 1 means this event has the second highest energy in the group it belongs to.
- Time of detection in picoseconds (for the event in the trigger region of higher ID number).
- Energy of the pulse, for the event in the trigger region of higher ID number (in QDC mode, in arbitrary units of charge; in TOT mode, in nanoseconds). If the energy calibration file (see section 5.8) exists and is referenced in the reference configuration file, the energy value will also have arbitrary units of charge.

- Absolute channel ID, for the event in the trigger region of higher ID number (see appendix A).
- “mh_n2”, same as “mh_n1”, but for the coincidence event in the trigger region of lower ID number.
- “mh_j2”, same as “mh_j1”, but for the coincidence event in the trigger region of lower ID number.
- Time of detection in picoseconds (for the event in the trigger region of lower ID number).
- Energy of the pulse, for the event in the trigger region of lower ID number (in QDC mode, in ADC units; in TOT mode, in nanoseconds).
- Absolute channel ID, for the event in the trigger region of lower ID number (see appendix A).

By default, only the the event with highest charge of each coincidence group (mh_j = 0) is written in the output file. In order to write all the events in both groups for one coincidence, the option “–writeMultipleHits” should be used specifying the maximum number of hits to the maximum possible value. An example of the output data stored in “data_folder/data_coincidences.txt” is (only writing the maximum charge events of each coincidence group):

```
...
16 0 746540679252 17.219238 280 2 0 746540688390 2.190071 115
21 0 746542092613 13.789963 289 7 0 746542093405 14.952671 115
14 0 746543556436 18.033173 278 17 0 746543557528 11.129692 122
18 0 746559438500 16.535572 283 27 0 746559437824 20.985302 87
7 0 746582738153 8.471455 270 8 0 746582738245 10.569595 101
3 0 746639786672 3.113194 260 21 0 746639784992 15.615772 109
16 0 746653183836 17.341839 278 21 0 746653184244 19.260880 119
6 0 746743348031 6.681160 302 13 0 746743346465 17.183113 97
2 0 746752823535 1.654106 270 14 0 746752821572 10.654236 98
15 0 746785502951 15.923786 279 10 0 746785503584 14.865990 122
31 0 746791064825 23.305775 267 9 0 746791065377 8.829895 73
21 0 746793932627 16.414318 263 16 0 746793933032 13.508621 99
19 0 746913280769 18.704937 271 7 0 746913281535 6.642433 110
10 0 746921420200 17.198643 275 14 0 746921421380 12.888512 125
9 0 746922094190 17.683727 275 14 0 746922095552 9.447754 123
24 0 746930656137 19.851555 269 8 0 746930657504 5.457588 107
14 0 746976127033 11.758198 283 2 0 746976128004 3.096554 106
10 0 747018393456 8.486469 279 14 0 747018393842 12.628952 126
2 0 747023540299 3.772789 274 5 0 747023541428 3.700508 123
18 0 747036209898 16.228996 285 1 0 747036212717 3.591759 127
15 0 747036694818 18.114315 275 5 0 747036697098 4.196793 121
26 0 747135818329 12.057774 288 33 0 747135817700 19.561619 65
20 0 747165260554 11.689510 258 14 0 747165260386 14.569408 110
8 0 747186019605 16.644855 273 33 0 747186019647 21.217892 100
4 0 747203593353 7.317780 274 13 0 747203592466 24.086327 78
...
```

Again, using the option “–writeBinary”, the output file will be a compact binary with 10 variables per event and the data format will be: 8-bit unsigned integer (mh_n1), 8-bit unsigned integer (mh_j1), 64-bit integer (time in picoseconds), 32-bit floating point (ToT or Energy) and 32-bit integer (channel

ID), 8-bit unsigned integer (mh_n2), 8-bit unsigned integer (mh_j2), 64-bit integer (time in picoseconds), 32-bit floating point (ToT or Energy) and 32-bit integer (channel ID). The full C code structure for each event is:

```

1 struct Event {
2     uint8_t mh_n1;
3     uint8_t mh_j1;
4     long long time1;
5     float e1;
6     int id1;
7     uint8_t mh_n2;
8     uint8_t mh_j2;
9     long long time2;
10    float e2;
11    int id2;
12 };

```

A third output data type is possible with the application “convert_raw_to_coincidence”, consisting of a ROOT file with the data in the form of a ROOT TTree named “data” with one coincidence event per entry. There will be two sets of “time”, “channelID”, “tac”, “tot”, “energy”, “mh_n”, “mh_j”, “xi”, “yi”, “x”, “y”, “z”, one for each event in a coincidence, as branches of the TTree “data”.

Several other command line options are available to control the filtering of coincidence events during processing and when writing to output file. Option “--help” prints to screen all the arguments and flags for “convert_raw_to_coincidence”.

5.3.1 Compact data formats

As with the group application, two additional output data formats were added to the coincidence application with the introduction of software version v2023.07.24. The new formats were designed to enhance the compactness of the output data structure and reduce the size of processed data files. However, the original three data formats are still available to ensure compatibility with older reading programs.

The new formats apply changes to the output structure of text and binary options, and can be selected using the flags “--writeTextCompact” and “--writeBinaryCompact”. The first will generate a text file where each coincidence is described with an header indicating the values of the number of events to be written for both groups in coincidence, followed by time, charge and channelID information for events from the first group (in order of decreasing charge) and the same for events of the second group (also with decreasing charge).

Below is an example for two coincidence groups where all registered hits are written to file (using the option “--writeMultipleHits 64”):

```

...
10 9
710411967981 14.454121 992
710411967337 10.984436 896
710411968298 9.169872 991
710411967696 8.478249 910
710411970919 2.807804 989
710411971209 2.299446 897
710411972818 1.489227 1011
710411973569 1.470882 903
710411979116 1.206451 906

```

```

710411978443 0.921841 997
710411933198 12.068851 110
710411931310 10.095474 16
710411936288 4.195087 109
710411934954 3.692257 106
710411934076 2.956257 18
710411935929 2.410999 17
710411935334 1.007076 61
710411935720 0.843014 24
710411939479 0.701862 107
9 11
710617182491 13.963394 970
710617183114 10.974209 935
710617183411 7.192448 998
710617183938 4.871899 972
710617183862 4.719608 943
710617185159 3.115463 987
710617185029 2.598240 942
710617187005 2.114838 939
710617192895 0.878166 976
710617182542 16.432434 95
710617181668 14.607140 14
710617184510 6.668270 100
710617182829 5.657436 26
710617186431 2.838490 96
710617186020 1.716156 0
710617187659 1.622623 116
710617194245 1.414032 97
710617190044 1.032516 7
710617187709 0.985535 27
710617191643 0.930176 115
...

```

As for the binary compact form, it uses the same principle writing an header value for each coincidence group composed of a structure with two 8-bit unsigned integers (`uint8_t`) describing the number of events to be written for each group in coincidence. The events for each group are sequentially written after the header using the same C code structure used for "single" events:

```

1 struct Event {
2     long long time;
3     float e;
4     int id;
5 };

```

An example of a C code snippet for reading such format is:

```

1 #include <stdio.h>
2
3 struct Event {
4     long long time;
5     float e;
6     int id;

```

```

7 } __attribute__((__packed__));
8
9 struct CoincidenceGroupHeader {
10     uint8_t nHits1;
11     uint8_t nHits2;
12 } __attribute__((__packed__));
13
14 Event ev;
15 CoincidenceGroupHeader header;
16
17 FILE * dataFile = fopen("data_folder/data_coinc.ldat", "rb");
18
19 while(!feof(dataFile)) {
20     fread(&header, sizeof(header), 1, dataFile);
21     Event coinc1[header.nHits1];
22     fread(coinc1, sizeof(Event), header.nHits1, dataFile);
23     Event coinc2[header.nHits2];
24     fread(coinc2, sizeof(Event), header.nHits2, dataFile);
25 }

```

5.4 Event filtering parameters

The applications described in sections 5.3 and 5.2 use some filtering parameters, which can be changed in the “[sw_trigger]” section in “config.ini” (all values provided are default). These are:

- “group_min_energy” - The minimum energy of an event to be considered for a multi-hit group of events. Units are in nanoseconds (TOT mode) or arbitrary units (QDC mode). Used both in “convert_raw_to_group” and “convert_raw_to_coincidence”.
- “group_max_energy” - The maximum energy of an event to be considered for a multi-hit group of events. Units are in nanoseconds (TOT mode) or arbitrary units (QDC mode). Used both in “convert_raw_to_group” and “convert_raw_to_coincidence”.
- “group_max_distance” - The maximum geometrical distance between two events to be considered for a multi-hit group of events. The distance is computed from the coordinates for each channel as defined in the last three columns of channel map - “map_channel.tsv” (see section). Used both in “convert_raw_to_group” and “convert_raw_to_coincidence”.
- “group_time_window” - The time window (in clock period units) for events to be considered as a multi-hit group of events. Used both in “convert_raw_to_group” and “convert_raw_to_coincidence”.
- “coincidence_time_window” - The time window (in clock period units) for 2 multi-hit groups of events to be considered in coincidence when using “convert_raw_to_coincidence”.

5.5 Additional output options

The three processing application described in the previous sections have 3 additional options (2 of them useful when handling large raw files):

- Using the option “--timeref REF”, allows the user to chose which time reference should be taken into account as the starting moment ($t=0$) for the output event timestamps. There are 4 options available for REF, which are:
 - sync - Corresponds to the moment when the synchronization signal was sent to all connected ASICs. This is the default time reference if “--timeref” is not explicitly used.
 - user - The moment of raw data file creation, coincident with manual acquisition start in the GUI or using the “wait-on” fifo to start data acquisition.
 - wall - Uses UNIX epoch time for all timestamps, making the time reference to be January 1st, 1970. Due to the large values for timestamps this option should not be used if the output data format is of type “text”.
 - step - This option can be used when the data acquisition is part of a parameter scan as described in section 4.2.2. The time reference for each parameter will be 0 for the first internal frame ID generated for each parameter.
- By using the flag “--writeFraction N”, where N is a value between 0 and 100 (representing a percentage), the programs process the entire raw data file but only write a fraction of the corresponding output events. This option is beneficial when dealing with large data sets, as it allows for a reduction in the size of the output files while still retaining a representative sample of the events. For example, if “--writeEventFraction 50” is used, only 50% of the events will be written to the output files.
- When the flag “--splitTime T” is activated, where T represents a time interval in seconds, the output data is divided into multiple files. Each file corresponds to a specific time acquisition interval defined by T. This option is useful for managing large data sets, as it enables the organization of data into smaller, more manageable chunks. For instance, if “--splitTime 10” is used, the output data will be split into multiple files, each containing events from a 10-second time interval.

5.6 Hardware trigger software simulation

For each of the 3 applications described above, it is possible to enable an optional flag in order to simulate the effect of the hardware trigger, using the parameters defined in the “[hw_trigger]” section of “config.ini” and described in section 4.2.3. It is possible to check in the report of each of these apps the number (and percentage) of events that passed the HW Trigger simulation, indicating how many events will be transmitted by the hardware trigger for the settings defined in “config.ini”. If using the singles application, the output file will contain a very reliable estimate of the raw data transmitted by the hardware trigger filter. In order to use this option, simply add the flag “--simulateHwTrigger” when processing the data raw (singles, groups, or coincidences). The raw file to be processed must be obtained with the hardware trigger disabled and under the same conditions (such as source activity, position, and temperature) that are required for the filter to operate. Due to the nature of the simulation, the activation of this option will significantly increase the running time of the processing application.

5.7 Time offset (or skew) calibration

Although all timestamps produced by each independent TOFPET2 channel are perfectly synchronized with the reference clock, their absolute value can present relative offsets or skews between different channels. These offsets are caused by several factors, such as differences in cable and internal lengths, among others.

The best way to correct the internal offsets is to perform an empirical calibration (obtained in typical measurement conditions), using a setup where the time difference between different channels is known. For example, it using PET detector modules (SIPM + Crystals arrays), knowing the expected time difference for each line of response. In non-PET applications, using only SIPM arrays, one can shine all SIPM pixels with a pulsed laser beam, and expect for the time difference between any given two channels to be 0.

Regardless of the method used to determine the time offsets, once the user has determined a time skew/offset value for each channel, they can be provided as an input calibration table to the data processing software (singles, or coincidences described in the next sections), allowing for the time stamps to be corrected when obtained in similar setup conditions. In order to do so, the user just needs to create a table and save it as “time_offset_calibration.tsv” in the working directory folder (WDF). The table should have the following format (offsets in picoseconds):

```
#portID slaveID chipID channelID tOffset (ps)
0 0 0 1 5.5732050e+02
0 0 0 2 7.8216449e+02
0 0 0 3 1.3669454e+03
0 0 0 4 7.4608004e+02
0 0 0 5 1.3020162e+03
...
0 0 1 49 1.0240485e+03
0 0 1 50 7.4695925e+02
0 0 1 51 1.7604513e+03
0 0 1 52 1.2218682e+03
0 0 1 53 1.4113197e+03
0 0 1 54 1.3031210e+03
...
```

5.8 QDC charge measurement linearisation / Energy Calibration (Optional)

The TOFPET2 QDC pre-amplifier has a slightly non-linear response over its nominal dynamical range. When measuring the full energy deposition of gamma-rays using scintillating crystals and SIPMs there are two effects contributing to the non-linearity of the charge (or energy) measurement: SiPM saturation and the (small) non-linear response of the TOFPET2 QDC pre-amplifier. If desired, one can use empirical ways to correct such overall non-linearity, for example, by using data obtained with different gamma-ray emitting isotopes, which can show spectra with photopeaks of known energies. As an example, one using ^{22}Na (511 keV), ^{137}Cs (32 and 662 keV) and ^{133}Ba (31 keV, 81 keV and 356 keV), one can effectively linearise the measurements by fitting the Energy (E) vs charge (Q in arbitrary units, as provided by the data processing software) with an appropriate parametric function, such as:

$$E = P_0 \cdot p_1^{Q^{P_2}} + P_3 \cdot Q - P_0 \quad (5.1)$$

In many situations, the user may not have such isotopes available in order to calibrate every ASIC channel independently. However, PETsys Electronics has verified that using the above function with a fixed parameterization $(P_0, P_1, P_2, P_3) = (8.00000, 1.04676, 1.02734, 0.31909)$ allows to approximate

the non-linear response of the ASIC's QDC for all channels, up to a level of 2-3% on the energy resolution of the 511 keV photopeak. In order to apply this "universal" linearisation, the user can create a simple table using the command (assuming the working data folder is "data"):

```
/make_simple_energy_calibration_table --config data/config.ini  
-o data/energy_calibration.tsv
```

The calibration table, whether created with the above script or created by the user individually for each channel, can be provided to the software to linearise/calibrate the charge measurement when processing data (see Chapter 5). To do so, just open the reference configuration file "config.ini", remove the comment on the line referring to "energy_calibration_table" and save.

When an energy calibration file exists in the WDF and is referenced in the configuration file, the energy of the pulse as output of any of the data processing applications (for any chosen format) will correspond to the application of equation 5.1 to the measured charge. If using the universal parameterization above, the resulting charge units are also arbitrary.

5.9 Raw to raw application

It was mentioned above that the raw data (time and energy) originated from the ASIC and FPGA needs to be corrected with calibration functions, hence the need for processing applications. Nevertheless, as a debugging tool to check if there is some problem in the raw data, the user may want to check this data before the application of any calibration. To do so, and since the raw data is in binary format, PETSYS Electronics also provides an application that converts the raw data into a ROOT TTree format.

In order to use it, just type:

```
./convert_raw_to_raw --config data_folder/config.ini  
-i data_folder/data_prefix -o data_folder/data_raw.root
```

The full TTree structure of the output data is:

- "step1" and "step2" are the values of step1 and step2 from the acquisition with multiple parameters. If no scan is performed, both this variables are set to 0.
- "channelID" is the absolute channel ID of the event (see appendix A).
- "tacID" is the TAC number for this event, ranging from 0 to 3.
- "tcoarse" is the coarse time stamp measured at the t1 discriminator, in clock periods, from 0 to 1023.
- "tfine" is the fine time stamp measured by the TDC at the t1 discriminator, in ADC units, typically ranging from 200 to about 400.
- "ecoarse", in TOT acquisition mode, is the coarse time stamp measured at the E discriminator, in clock periods, from 0 to 1023; in QDC acquisition mode, is the coarse time stamp corresponding to the charge integration ending, in clock periods, from 0 to 1023.
- "efine" in TOT acquisition mode, is the fine time stamp measured by the TDC at the E discriminator, in ADC units, typically ranging from 200 to about 400; in QDC acquisition mode, is the charge integration value, in ADC values.

- “frameID” identifies the frame of data to each the event belongs. Each frame contains 1024 clock periods.

5.10 Data Processing with GUI

The above sections described the available processing applications and output options. All these functionalities and options are available in the GUI, where the user can choose the raw file name (or prefix) to be processed, the output file name, the event type of the processed file (“Singles”, “Coincidences”, “Groups” or “Raw”) and the output data format: “Binary”, “Text” or “ROOT”. The user can also choose two other options: “Fraction”, defining the fraction of the raw data that will be effectively processed (from 0 to 100%) and the “Number of Hits”, which is only applicable when processing to Coincidences or Grouped events, as described above.

5.11 Real-time data processing

With the release of v2025.05.21, a new feature was introduced to allow the processing of raw data (as documented above in this chapter) while performing the data acquisition, hereafter referred to as real-time data processing. Such feature provides the user with enhanced flexibility for the data acquisition output, avoiding the need for lengthy run-times for post-processing of raw data, specially relevant for long acquisition durations, a large number of channels and/or high data rates, which typically result in very large raw data file sizes. When using this optional feature, the user can choose to write only the processed final output data (singles, groups, or coincidences) and use filtering parameters described in section 5.4, to further reduce the size of the output files needed for the final application. Such is the case of list-mode data for image reconstruction in PET applications.

In order to activate this feature, use the flag “--enable-realtime-processing” when acquiring data with the “acquire_sipm_data” program. In addition to activating the flag, the user should also define the following required variables, in order to obtain the desired output data:

- “--output-type TYPE” - Select the type of output processed data where “TYPE” can be: “singles”, “groups”, “coincidences” or “raw” (see sections 5.1, 5.2, 5.3).
- “--output-format FORMAT” - Select the data format of the output processed where “FORMAT” can be : “text”, “textCompact”, “binary”, “binaryCompact”, “root” (see sections 5.1, 5.2, 5.3).

The data acquisition script will use these to process the data accordingly, while adding automatically a suffix and file extension to the file output name. Moreover, optional variables described in the aforementioned sections are available when “--enable-realtime-processing” is set. These are:

- “--writeRawData” - Chose this flag in order to also write the raw data file in addition to the processed data file. This is useful to be able to re-process and filter data with different settings, although it can reduce the maximum total rate that can be processed without data loss.
- “--writeFraction FRACTION” - The total fraction of events to write in the processed data file (see section 5.5).
- “--timeRef REF” - Select the time reference for the event timestamps (see section 5.5).
- “--writeMultipleHits N” - The maximum number of events in a multi-hit group to be written to file (if chosen output type is “groups” or “coincidences”).

When using the GUI, just activate the switch to enable the real-time data processing, and select the “Save Raw Data” checkbox in order to write the raw data file in addition to the processed file. When the user starts acquiring, the processing will be made according to the settings defined in the “Data Processing” section.

IMPORTANT: When using the real-time processing feature, there is a maximum incoming data rate that can be processed without data loss. This maximum rate depends on the computer hardware (number of CPU cores, disk speed, CPU frequency) as well as output settings (format and type, along with possible group and coincidence settings). For this reason, whenever “–enable-realtime-processing” is activated, the acquisition script performs a brief preliminary acquisition. If data loss is detected, it reports the estimated fraction of data expected to be lost and requests confirmation before proceeding with the full acquisition. If data loss occurs and is undesirable, the user should either adjust the settings to reduce CPU and disk load or use only the offline data acquisition, which only writes the raw data file.

Appendix A

Absolute ID of Channels, ASICs and HV DAC channels

The software and firmware assign an absolute ID number to each ASIC, channel, and HV DAC channel. The ID numbering is related directly to the FEB/D port where a given TOFPET2 FEM is plugged.

The absolute IDs are constructed in the following way:

$$\text{Absolute Channel ID} = 64 \times \text{chipID} + \text{channelID}$$

where:

- **chipID:** The ASIC ID, related to the position where it is connected in the FEB/D. This number can range from 0 to 15, where 0 and 1 correspond to the ASICs belonging to the Front End Module connected in port F1 of the FEB/D, 2 and 3 to the ones connected in port F2 and so on up to port F8. In a given Front End Module, ASIC 0 is the one belonging to the FEB/A connected to J1 of the FEB/I and ASIC 1 is the one belonging to the FEB/A connected to J2 of the FEB/I.
- **channelID:** The channel ID inside the ASIC, from 0 to 63. In order to know exactly what input is connected to a given ASIC channel, please refer to tables 2 and 3 of the “TOFPET2 ASIC SIPM Readout System - Hardware user Guide”.

Regarding the Bias DAC channel identification, used in the Bias calibration file and Bias settings file, it follows the same logic for identification of the FEB/D, using “portID” and “slaveID” and a channelID that is related to the specific HV DAC channel within the FEB/D. In order to know where each channelID connects to, the user may refer to the tables present in the folder “TOFPET2 > Mapping tables” on the Client Package Folder.